

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Ứng dụng Mô hình ngôn ngữ lớn và Tác tử AI trong
xây dựng hệ thống hỗ trợ học tập

NGUYỄN KHẮC ĐIỆP

diep.nk225806@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Ngô Văn Linh

Ngành: Công nghệ thông tin

Trường: Công nghệ thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

Ứng dụng Mô hình ngôn ngữ lớn và Tác tử AI trong
xây dựng hệ thống hỗ trợ học tập

NGUYỄN KHẮC ĐIỆP
diep.nk225806@sis.hust.edu.vn

Chương trình đào tạo: Công nghệ thông tin Việt-Nhật

Giảng viên hướng dẫn: TS. Ngô Văn Linh _____

Chữ kí GVHD

Khoa: Công nghệ thông tin Việt-Nhật

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2022

LỜI CẢM ƠN

Để hoàn thành đồ án này, em xin gửi lời cảm ơn chân thành đến quý thầy cô đã tận tâm hướng dẫn và trang bị kiến thức chuyên môn quý báu. Con xin cảm ơn gia đình vì luôn là chỗ dựa tinh thần vô giá. Cảm ơn những người bạn đồng hành và đặc biệt là người yêu đã luôn thấu hiểu, san sẻ áp lực và động viên em trong suốt thời gian qua. Và cuối cùng, tôi muốn gửi lời cảm ơn đến chính bản thân mình vì đã không bỏ cuộc, luôn chăm chỉ, kiên định và quyết tâm nỗ lực vượt qua mọi thử thách để hoàn thiện công việc với kết quả tốt nhất. Sự đồng hành của tất cả mọi người là động lực to lớn giúp em đạt được cột mốc này. Xin trân trọng cảm ơn!

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh chuyển đổi số giáo dục, học sinh và giáo viên Trung học Phổ thông (đặc biệt môn Tin học) đối mặt với nhiều thách thức. Học sinh cần môi trường học tương tác, còn giáo viên mất nhiều thời gian soạn giáo án, sinh bài tập và chấm điểm. Mặc dù các mô hình ngôn ngữ lớn (LLM) mang lại tiềm năng lớn, chúng vẫn gặp rào cản về việc sinh thông tin sai lệch (hallucination) và không bám sát chương trình chuẩn. Từ đó, bài toán đặt ra là xây dựng một trợ lý ảo giáo dục thông minh có khả năng xử lý tự động, chính xác các tác vụ dựa trên nguồn tri thức chuẩn mực.

Nghiên cứu ứng dụng kiến trúc tìm kiếm và sinh văn bản (RAG) kết hợp phương pháp điều phối đa tác nhân (Multi-Agent LLM). Cụ thể, cơ sở dữ liệu vector được xây dựng từ sách giáo khoa chuẩn. Quá trình truy xuất dữ liệu áp dụng tìm kiếm lai (từ khóa và ngữ nghĩa), tối ưu bằng thuật toán Reciprocal Rank Fusion và mạng Cross-Encoder để xếp hạng lại nhằm trích xuất chính xác ngữ cảnh. Đồng thời, hệ thống bóc tách, đánh giá và định tuyến đa ý định thông qua bộ điều khiển luồng trung tâm (Pipeline Controller). Các dịch vụ miền sau đó xử lý lệnh song song cho các tác vụ như hỏi đáp, sinh bài tập đa định dạng, chấm điểm tự luận theo tiêu chí và tạo cấu trúc bài giảng.

Thử nghiệm trên 246 mẫu kiểm thử bằng khung đo lường RAGAS cho kết quả rất khả quan. Hệ thống đạt tính trung thực (Faithfulness) 0.975 giúp khắc phục gần như hoàn toàn hiện tượng ảo giác, độ phủ ngữ cảnh (Context Recall) đạt 0.959 và mức độ liên quan (Answer Relevancy) đạt 0.857. Tốc độ phản hồi trung bình của toàn hệ thống duy trì ổn định ở mức 6,5 giây cho mỗi truy vấn phức tạp.

Nghiên cứu cung cấp một giải pháp công nghệ giáo dục (EdTech) hiệu quả, hỗ trợ tự động hóa một số quy trình dạy và học môn Tin học THPT. Đo thời, kết quả chứng minh sự thành công của kiến trúc lai RAG và Multi-Agent trong việc định hướng LLM, tạo nền tảng vững chắc để phát triển các hệ sinh thái học tập cá nhân hóa sâu rộng hơn.

Sinh viên thực hiện
(Ký và ghi rõ họ tên)

ABSTRACT

In the digital transformation of education, high school students and teachers face notable challenges in Computer Science. Students need interactive self-study environments, while teachers spend excessive time planning lessons, creating quizzes, and grading. Although Large Language Models (LLMs) offer high potential, they suffer critically from information hallucination and a lack of grounding to authorized curricula. Thus, this project proposes an automated, intelligent virtual educational assistant strictly bounded by standardized knowledge bases.

To resolve these challenges, the research introduces a system structured on a Retrieval-Augmented Generation (RAG) architecture orchestrated by a Multi-Agent LLM approach. A vector database was built indexing textbook content from grades 10 to 12. The retrieval pipeline employs a Hybrid Search strategy optimized by Reciprocal Rank Fusion and a Cross-Encoder model for precise Vietnamese document reranking. Driven by a Thin Pipeline Controller that supports dynamic multi-intent routing, domain services parallelly handle complex workflows. These include standard knowledge queries, context-aware quiz generation in varying formats, criteria-based semantic answer grading, and structured lesson plan generation.

Evaluated using the RAGAS framework across 246 comprehensive test instances, the system attained outstanding metrics. It recorded a Faithfulness score of 0.975 (virtually eliminating hallucinations), a Context Recall score of 0.959, and an Answer Relevancy score of 0.857. Furthermore, the framework demonstrated robust performance, achieving an average response latency of 6.5 seconds per multi-intent query.

Ultimately, this thesis presents a scalable EdTech solution that pragmatically automates pedagogical steps for High School Computer Science. It empirically affirms the capability of hybrid retrieval architectures coupled with multi-agent orchestration to mitigate language model hallucinations structurally. This establishes a firm foundation for deploying proactive, personalized learning ecosystems in Vietnam.

Danh mục ký hiệu và chữ viết tắt

Ký hiệu	Ý nghĩa tiếng Anh	Ý nghĩa tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
API	Application Programming Interface	Giao diện lập trình ứng dụng
BM25	Best Matching 25	Thuật toán đối sánh tốt nhất 25
BPE	Byte-Pair Encoding	Một thuật toán mã hóa từ vựng
CD		Cánh điều (Sách giáo khoa)
CoT	Chain-of-Thought	Chuỗi tư duy
HyDE	Hypothetical Document Embeddings	Biểu diễn tài liệu giả định
IDF	Inverse Document Frequency	Tần suất nghịch đảo tài liệu
JSON	JavaScript Object Notation	Định dạng dữ liệu văn bản
KNTT		Kết nối tri thức (Sách giáo khoa)
LLM	Large Language Model	Mô hình ngôn ngữ lớn
NLP	Natural language processing	Xử lý ngôn ngữ tự nhiên
Q&A	Question and Answer	Câu hỏi và câu trả lời
RAG	Retrieval-Augmented Generation	Tạo sinh tăng cường bằng truy xuất
RRF	Reciprocal Rank Fusion	Kết hợp thứ hạng nghịch đảo
SGK		Sách giáo khoa
TF	Term Frequency	Tần suất xuất hiện của từ

Danh mục thuật ngữ chuyên ngành

Thuật ngữ	Giải thích ý nghĩa
Agent	<i>(Tác tử)</i> - Hệ thống tự trị có lỗi là trí tuệ nhân tạo, có khả năng diễn dịch yêu cầu, lập suy luận và sử dụng công cụ để thực thi hành động.
Breadcrumb	<i>(Đường dẫn phân cấp)</i> - Chuỗi ngữ cảnh ngắn ghi lại vị trí của một chunk trong cấu trúc tài liệu (dạng chủ đề > bài > mục), được ghép vào nội dung để phân biệt các chunk có câu chữ tương tự nhưng thuộc bài khác nhau.
Chunk	<i>(Phân đoạn)</i> - Một đoạn văn bản có ý nghĩa hoàn chỉnh được cắt nhỏ từ tài liệu gốc, có kích thước vừa vặn với bộ nhớ của mô hình ngôn ngữ.
Chunking	<i>(Phân mảnh dữ liệu)</i> - Kỹ thuật chia nhỏ các tài liệu quy mô lớn thành các chunk nhằm thuận tiện cho việc lập chỉ mục và tìm kiếm thông tin.
Context Window	<i>(Cửa sổ ngữ cảnh)</i> - Giới hạn tối đa số lượng từ (hoặc token) mà một LLM có thể tiếp nhận và ghi nhớ trong một phiên xử lý duy nhất.
Embedding	<i>(Biểu diễn Vector/Nhúng)</i> - Kỹ thuật mã hóa ngôn ngữ tự nhiên thành các vector số học có độ phân giải cao, giúp máy tính có thể tính toán đo lường mức độ tương đồng ý nghĩa.
Hallucination	<i>(Hiện tượng Ảo giác)</i> - Trạng thái mà mô hình ngôn ngữ lớn đưa ra những câu trả lời có hệ thống về ngoài rất tự tin nhưng hoàn toàn sai sự thật hoặc thiếu căn cứ.
Metadata	<i>(Siêu dữ liệu)</i> - Các thẻ thông tin đi kèm (như Tên bài học, Chương, Chủ đề) nhằm hỗ trợ và cung cấp căn cứ nguồn gốc cho các chunk dữ liệu.
Multi-Agent	<i>(Đa tác tử)</i> - Một hệ thống phức hợp nơi nhiều tác tử (agent) hoạt động song song hoặc phối hợp nhằm giải tỏa áp lực của một tác vụ quá lớn.
Prompt	<i>(Chỉ thị)</i> - Câu lệnh hoặc ngữ cảnh đầu vào hướng dẫn mô hình ngôn ngữ lớn thực hiện trả lời, thiết lập phong cách hay xuất ra bằng định dạng nhất định.

Reranker	<i>(Bộ xếp hạng lại)</i> - Mô hình máy học nâng cao làm nhiệm vụ rà soát, tinh chỉnh thứ tự các kết quả tìm ráp ban đầu để đẩy văn bản có liên đới chặt chẽ nhất lên cực đỉnh.
Retriever	<i>(Bộ truy xuất)</i> - Thành phần thuật toán thực hiện tìm kiếm, quét qua toàn bộ cơ sở dữ liệu Vector để lấy ra cụm tài liệu thô gần khớp với truy vấn người dùng.
Token	<i>(Đơn vị cơ sở)</i> - Thành phần nhỏ nhất của một đoạn văn bản (có thể là một từ, âm tiết, hoặc cụm ký tự) mà máy tính có thể thể tách ra để xử lý.
Vector Database	<i>(Cơ sở dữ liệu Vector)</i> - Cơ sở dữ liệu chuyên dụng được thiết kế nhằm lưu giữ và truy vấn tốc độ cao đối với dữ liệu kiểu vector đa chiều.
Underthesesa	<i>(Thư viện NLP tiếng Việt)</i> - Một bộ công cụ mã nguồn mở phụ vụ mạnh mẽ cho nhiều bài toán xử lý ngôn ngữ tự nhiên tiếng Việt, nổi bật là tách từ (Word Segmentation).
Word Segmentation	<i>(Tách từ)</i> - Kỹ thuật phân tách chuỗi văn bản thành hệ thống các từ vựng hợp cấu trúc, đặc biệt quan trọng với ngôn ngữ không có định danh ranh giới sẵn bằng khoảng trắng (ví dụ từ ghép tiếng Việt).

MỤC LỤC

Danh mục ký hiệu và chữ viết tắt.....	i
Danh mục thuật ngữ chuyên ngành	ii
CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Mục tiêu và phạm vi đề tài.....	2
1.3 Định hướng giải pháp.....	3
1.4 Bố cục đồ án	4
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT, KHẢO SÁT VÀ ĐỀ XUẤT GIẢI PHÁP.....	5
2.1 Phạm vi nghiên cứu	5
2.2 Tổng quan về bài toán	6
2.2.1 Bài toán chatbot giáo dục.....	6
2.2.2 Hệ thống hỏi đáp dựa trên tài liệu	7
2.2.3 Thách thức đặc thù của dữ liệu SGK Tin học	7
2.3 Khảo sát các giải pháp liên quan	8
2.3.1 Các nhóm giải pháp hiện có	8
2.3.2 Bảng so sánh các nhóm giải pháp	9
2.3.3 Nhận xét và định vị giải pháp đồ án	10
2.4 Cơ sở lý thuyết và hướng tiếp cận.....	10
2.4.1 Mô hình ngôn ngữ lớn và hạn chế khi dùng trực tiếp	10
2.4.2 Embedding và tìm kiếm ngữ nghĩa.....	11
2.4.3 Retrieval-Augmented Generation.....	11
2.4.4 Adaptive retrieval, reranking và query rewriting.....	12
2.4.5 Multi-intent và điều phối yêu cầu học tập	12

2.4.6 Đánh giá hệ thống RAG.....	13
2.4.7 Khoảng trống và hướng tiếp cận của đề án	13
2.5 Kết chương.....	13
CHƯƠNG 3. THIẾT KẾ GIẢI PHÁP VÀ PHƯƠNG PHÁP ĐÁNH GIÁ.	15
3.1 Tổng quan kiến trúc hệ thống.....	15
3.1.1 Bài toán và yêu cầu	15
3.1.2 Kiến trúc tổng thể và luồng xử lý một yêu cầu	16
3.1.3 Công nghệ nền.....	17
3.2 Nền tảng tri thức: tiền xử lý, chunking và lập chỉ mục	17
3.2.1 Nguồn SGK, cấu trúc phân cấp và lược đồ metadata.....	19
3.2.2 Chunking theo tiêu đề.....	20
3.2.3 Biểu diễn vector tiếng Việt và lưu trữ Qdrant.....	22
3.3 Cơ chế truy hồi ngữ cảnh (RAG core).....	23
3.3.1 Truy hồi lai BM25 và dense, hợp nhất bằng Reciprocal Rank Fusion.....	23
3.3.2 Xếp hạng lại bằng cross-encoder	24
3.3.3 Truy hồi thích ứng (Adaptive RAG).....	25
3.4 Tầng điều phối tác tử.....	27
3.4.1 Sự cần thiết của tầng điều phối tác tử	27
3.4.2 Định tuyến ý định bằng mô hình ngôn ngữ lớn.....	27
3.4.3 Làm giàu ngữ cảnh hội thoại và viết lại truy vấn	29
3.4.4 Quản lý trạng thái hội thoại và vòng đời session.....	29
3.4.5 Lập kế hoạch hành động và thực thi đa hành động.....	31
3.5 Tầng sinh nội dung đa tác tử	33
3.5.1 Mô hình Supervisor–Specialist và lý do lựa chọn	33
3.5.2 Giao thức giao tiếp và cơ chế điều phối giữa các tác tử	33

3.5.3 Các tác tử chuyên biệt trong pipeline sinh nội dung	35
3.5.4 Tổng hợp kết quả, kiểm soát chất lượng và con người trong vòng lặp	36
3.5.5 Tái sử dụng kiến trúc cho các tác vụ sinh nội dung	37
3.6 Phương pháp đánh giá	38
3.6.1 Đánh giá truy hồi	38
3.6.2 Đánh giá chất lượng sinh	38
3.7 Kết chương.....	38
CHƯƠNG 4. CÀI ĐẶT, KIỂM THỬ VÀ KẾT QUẢ THỰC NGHIỆM....	39
4.1 Môi trường và dữ liệu thực nghiệm.....	39
4.2 Cấu trúc cài đặt hệ thống.....	40
4.3 Cài đặt pipeline xử lý tài liệu	40
4.4 Cài đặt pipeline hỏi đáp.....	41
4.5 Cài đặt retrieval, reranking và generation	41
4.6 Cài đặt benchmark và quy trình đánh giá	42
4.7 Kiểm thử chức năng	43
4.8 Kết quả đánh giá retrieval	44
4.9 Kết quả đánh giá generation và thảo luận	45
CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT.....	47
5.1 Điều phối request bằng thin orchestrator	47
5.1.1 Vấn đề cần giải quyết.....	47
5.1.2 Giải pháp	47
5.1.3 Đóng góp đạt được	48
5.2 Truy xuất lai và Adaptive RAG	48
5.2.1 Vấn đề cần giải quyết.....	48
5.2.2 Giải pháp	48

5.2.3 Đóng góp đạt được	49
5.3 Multi-intent routing và action planning	49
5.3.1 Vấn đề cần giải quyết.....	49
5.3.2 Giải pháp	49
5.3.3 Đóng góp đạt được	50
5.4 Tách quiz standalone khỏi content assessment	50
5.4.1 Vấn đề cần giải quyết.....	50
5.4.2 Giải pháp	50
5.4.3 Đóng góp đạt được	51
5.5 Content multi-agent cho slide và giáo án.....	51
5.5.1 Vấn đề cần giải quyết.....	51
5.5.2 Giải pháp	51
5.5.3 Đóng góp đạt được	51
5.6 Human-in-the-loop tại bước duyệt dàn ý	52
5.6.1 Vấn đề cần giải quyết.....	52
5.6.2 Giải pháp	52
5.6.3 Đóng góp đạt được	52
5.7 Quality path, session và trace	52
5.7.1 Vấn đề cần giải quyết.....	52
5.7.2 Giải pháp	52
5.7.3 Đóng góp đạt được	53
5.8 Tool layer như điểm mở rộng có kiểm soát	53
5.8.1 Vấn đề cần giải quyết.....	53
5.8.2 Giải pháp	53
5.8.3 Đóng góp đạt được	53
5.9 Kết chương.....	53

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	55
6.1 Kết luận.....	55
6.2 Hướng phát triển.....	55
TÀI LIỆU THAM KHẢO.....	58
PHỤ LỤC.....	60
A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP	60
A.1 Ngành học.....	61
A.2 Đánh dấu (bullet) và đánh số (numering)	61
A.3 Cách thêm bảng	62
A.4 Chèn hình ảnh	62
A.5 Tài liệu tham khảo	63
A.6 Cách viết phương trình và công thức toán học.....	63
A.7 Qui cách đóng quyển.....	63
B. ĐẶC TẢ USE CASE.....	65
B.1 Đặc tả use case “Thống kê tình hình mượn sách”	65
B.2 Đặc tả use case “Đăng ký làm thẻ mượn”	65

DANH MỤC HÌNH VẼ

Hình 3.1	Kiến trúc tổng thể của phương pháp đề xuất theo hướng phân tầng, trong đó lõi truy xuất tri thức là nền dùng chung cho các nhánh xử lý	16
Hình 3.2	Quy trình từ văn bản SGK đến tập chunk được lập chỉ mục, với chunking theo tiêu đề là bước trung tâm	18
Hình 3.3	Cơ chế truy hồi thích ứng của hệ thống với bộ phân loại truy vấn và các chiến lược truy xuất tương ứng	26
Hình 3.4	Khối phân tích ngữ cảnh và định tuyến ý định trong tầng điều phối	28
Hình 3.5	Cơ chế lập kế hoạch hành động ở tầng điều phối	31
Hình 3.6	Vòng thực thi đa hành động trong tầng điều phối theo chu trình quan sát – quyết định – hành động	32
Hình 3.7	Kiến trúc đa tác tử cho bài toán sinh slide và giáo án	35
Hình 3.8	Hai nhánh sinh nội dung và cơ chế hậu kiểm chất lượng của hệ thống	37
Hình A.1	Internet vạn vật	62
Hình A.2	Quy cách đóng quyển đồ án	64
Hình A.3	Quy cách đóng quyển đồ án	64

DANH MỤC BẢNG BIỂU

Bảng 2.1	So sánh các nhóm giải pháp cho bài toán chatbot giáo dục dựa trên SGK	9
Bảng 3.1	Phân bố số lượng chunk theo bộ sách và khối lớp	19
Bảng 3.2	Lược đồ metadata của một chunk	20
Bảng 3.3	Phân bố số lượng chunk theo loại nội dung sư phạm (type) .	20
Bảng 3.4	Ví dụ minh họa cách chunking theo tiêu đề trên một tài liệu ba tầng heading	21
Bảng 4.1	Môi trường phần mềm và các thành phần cốt lõi của hệ thống thực nghiệm	39
Bảng 4.2	Phân bố bộ benchmark retrieval theo bộ sách và khối lớp . . .	42
Bảng 4.3	Các ca kiểm thử chức năng chính của hệ thống trước khi chạy thực nghiệm	43
Bảng 4.4	Kết quả retrieval tổng thể trên 600 câu hỏi benchmark ở cấu hình no-rerank	44
Bảng 4.5	Kết quả retrieval no-rerank theo từng nhóm bộ sách và khối lớp	45
Bảng 4.6	Kết quả đánh giá generation theo RAGAS trên 246 mẫu	45
Bảng A.1	Table to test captions and labels.	62

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh cuộc cách mạng công nghiệp lần thứ tư đang diễn ra mạnh mẽ trên toàn cầu, chuyển đổi số đã trở thành một trong những ưu tiên hàng đầu của Việt Nam trong chiến lược phát triển quốc gia. Giáo dục được xác định là một trong những lĩnh vực trọng điểm trong quá trình chuyển đổi số, việc ứng dụng công nghệ thông tin và trí tuệ nhân tạo (AI) vào dạy và học không chỉ là xu hướng tất yếu mà còn là yêu cầu cấp thiết để nâng cao chất lượng giáo dục.

Môn Tin học tại trường trung học phổ thông (THPT) đóng vai trò đặc biệt quan trọng trong chương trình giáo dục Việt Nam. Với chương trình mới bao gồm hai bộ sách giáo khoa Cánh Diều và Kết Nối Tri Thức với Cuộc Sống cho các lớp 10, 11 và 12, nội dung bao phủ nhiều chủ đề chuyên môn từ khoa học máy tính, mạng máy tính đến an toàn thông tin. Tuy nhiên, việc giảng dạy và học tập môn học này vẫn đối mặt với nhiều thách thức đáng kể.

Về phía học sinh, các em thường thiếu một công cụ hỗ trợ tự học hiệu quả và bám sát nội dung sách giáo khoa. Các chatbot AI phổ quát hiện nay như ChatGPT, Claude hay Gemini tuy có khả năng trả lời câu hỏi nhưng không được tối ưu cho dữ liệu sách giáo khoa Việt Nam, dẫn đến tình trạng đưa ra thông tin không chính xác (hallucination). Điều này đặc biệt nguy hiểm trong bối cảnh giáo dục khi mà việc tiếp thu kiến thức sai lệch có thể ảnh hưởng nghiêm trọng đến quá trình học tập. Bên cạnh đó, học sinh cũng thiếu nguồn bài tập đa dạng và phản hồi tức thời được thiết kế chuyên biệt theo từng bài học.

Về phía giáo viên, việc biên soạn tài liệu giảng dạy như câu hỏi trắc nghiệm, bài tập hay cấu trúc slide bài giảng đòi hỏi rất nhiều thời gian và công sức. Trong khi đó, hầu hết các nền tảng e-learning hiện có tại Việt Nam chỉ đóng vai trò là kho lưu trữ và phân phối bài giảng số, chưa có khả năng tự động khởi tạo nội dung thông minh dựa trên yêu cầu của người dùng.

Từ những phân tích trên, nhu cầu xây dựng một hệ thống trợ lý ảo giáo dục ứng dụng công nghệ Tăng cường truy xuất dữ liệu (RAG) sử dụng dữ liệu nền tảng là sách giáo khoa trở nên cấp thiết. Hệ thống này cần đáp ứng đầy đủ các mục tiêu bao gồm tự động trả lời câu hỏi dựa trên nội dung SGK, sinh bài tập đa dạng định dạng, phát thảo cấu trúc slide bài giảng và hỗ trợ giáo viên trong công tác soạn giảng, qua đó tối ưu hóa công việc của cả người dạy lẫn người học.

1.2 Mục tiêu và phạm vi đề tài

Trong những năm gần đây, việc ứng dụng trí tuệ nhân tạo vào giáo dục đã thu hút sự quan tâm của nhiều nhà nghiên cứu và doanh nghiệp trên thế giới. Các mô hình ngôn ngữ lớn (LLM) như ChatGPT, Claude hay Gemini đã chứng minh khả năng ấn tượng trong việc tạo sinh văn bản và trả lời câu hỏi. Tuy nhiên, khi được sử dụng trong bối cảnh giáo dục cụ thể, các mô hình này bộc lộ những hạn chế đáng kể. Đầu tiên, chúng không được huấn luyện trên dữ liệu sách giáo khoa Việt Nam, dẫn đến thường xuyên đưa ra câu trả lời không chính xác hoặc không phù hợp với nội dung chương trình học. Thứ hai, hiện tượng hallucination - tức là sinh ra thông tin nhìn có vẻ đáng tin cậy nhưng hoàn toàn sai sự thật - là vấn đề nan giải mà các LLM hiện tại vẫn chưa thể khắc phục triệt để.

Đối với các nền tảng e-learning trong nước như VnEdu, HOC247, hay Moon, các hệ thống này chủ yếu cung cấp kho tài liệu số và video bài giảng được biên soạn sẵn. Một số nền tảng đã tích hợp tính năng quiz tự động nhưng chưa có khả năng sinh câu hỏi theo yêu cầu cụ thể của người dùng hoặc dựa trên nội dung bài học mới được cập nhật. Việc quản lý và cập nhật ngân hàng câu hỏi vẫn phải thực hiện thủ công, tốn kém nhiều thời gian và công sức.

Về mặt kỹ thuật, công nghệ Retrieval-Augmented Generation (RAG) đã được chứng minh hiệu quả trong việc cải thiện độ chính xác của LLM bằng cách kết hợp khả năng truy xuất tài liệu với khả năng sinh ngôn ngữ tự nhiên. Nhiều nghiên cứu đã áp dụng RAG vào các bài toán giáo dục và đạt được những kết quả khả quan. Tuy nhiên, phần lớn các ứng dụng này tập trung vào các ngữ cảnh sử dụng tiếng Anh và chưa có nhiều nghiên cứu cụ thể cho việc xây dựng hệ thống RAG dựa trên sách giáo khoa Việt Nam. Đặc biệt, việc kết hợp kiến trúc Multi-Agent với RAG để tạo ra hệ thống đa tác vụ (quiz + slide + scoring) vẫn còn là lĩnh vực tương đối mới và chưa được khai thác nhiều trong bối cảnh giáo dục Việt Nam.

So sánh giữa các giải pháp hiện có cho thấy rõ sự khác biệt về năng lực. Trong khi các chatbot AI phổ quát có khả năng đối thoại tự do nhưng không đảm bảo tính chính xác theo SGK, các nền tảng e-learning trong nước có nguồn dữ liệu chuẩn nhưng thiếu khả năng tự động sáng tạo nội dung. Đề tài này hướng tới việc kết hợp ưu điểm của cả hai hướng tiếp cận: đảm bảo tính bám sát sách giáo khoa đồng thời có khả năng sinh tự động các nội dung giáo dục đa dạng.

Dựa trên các phân tích và đánh giá ở trên, đề tài này xác định các hạn chế cần khắc phục bao gồm: thiếu công cụ AI tối ưu cho việc học Tin học THPT tại Việt Nam; chưa có giải pháp RAG nào tích hợp đầy đủ các tác vụ quiz, slide và scoring; và chưa có hệ thống nào kết hợp Multi-Agent với cơ chế Human-In-The-

Loop (HITL) để đảm bảo chất lượng đầu ra trong việc sinh slide bài giảng.

Mục tiêu tổng quát của đề tài là xây dựng hệ thống trợ lý ảo giáo dục hỗ trợ việc dạy và học môn Tin học tại trường trung học phổ thông, sử dụng nền tảng là sách giáo khoa Tin học lớp 10, 11 và 12 thuộc hai bộ sách Cánh Diều và Kết Nối Tri Thức.

Các mục tiêu cụ thể bao gồm: nghiên cứu và xây dựng hệ thống truy xuất kiến thức từ sách giáo khoa dựa trên kiến trúc RAG; thiết kế và phát triển module sinh câu hỏi và bài tập tự động với nhiều định dạng bao gồm trắc nghiệm, tự luận, điền khuyết và đúng/sai; xây dựng module sinh cấu trúc slide bài giảng với kiến trúc đa tác tử; phát triển module chấm điểm và phản hồi tự động dựa trên ngữ cảnh bài học; cuối cùng là đánh giá hiệu năng hệ thống thông qua các chỉ số đánh giá chuẩn.

1.3 Định hướng giải pháp

Để giải quyết các vấn đề đã xác định ở phần trên, đề tài này lựa chọn định hướng giải pháp dựa trên sự kết hợp giữa công nghệ Retrieval-Augmented Generation (RAG) và kiến trúc Multi-Agent. Đây là hai công nghệ tiên tiến trong lĩnh vực AI hiện nay, có khả năng bổ trợ lẫn nhau để tạo ra một hệ thống vừa đảm bảo tính chính xác vừa có khả năng xử lý đa tác vụ phức tạp.

Về nền tảng LLM, hệ thống sử dụng mô hình ngôn ngữ lớn của Google với ưu điểm là chi phí thấp, khả năng hỗ trợ tiếng Việt tốt và chế độ sinh output có cấu trúc ổn định. Mô hình này được đánh giá là phù hợp với mục tiêu của đề tài khi cần tối ưu hóa chi phí trong khi vẫn đảm bảo chất lượng đầu ra cần thiết.

Về thành phần truy xuất, thay vì phụ thuộc vào các thư viện đóng gói, đề tài tự xây dựng bộ máy truy xuất kết hợp giữa thuật toán BM25 và tìm kiếm ngữ nghĩa (Semantic Search). Hai luồng kết quả này được hợp nhất thông qua thuật toán Reciprocal Rank Fusion (RRF). Cách tiếp cận này mang lại sự chủ động trong việc tối ưu hóa thuật toán cho văn bản tiếng Việt và tránh phụ thuộc vào các thư viện có thể gây tốn kém tài nguyên không cần thiết.

Về kiến trúc điều phối, hệ thống thiết kế một Orchestrator tại lớp mức lệnh cơ sở thay vì sử dụng các framework có sẵn. Orchestrator này đóng vai trò là bộ xử lý mỏng, điều phối các thành phần xử lý bao gồm ContextAnalyzer, SessionManager, ActionPlanner và RAG Agent. Ưu điểm của cách thiết kế này là toàn bộ quy trình xử lý được tối ưu về tốc độ, đồng thời quá trình theo vết và debug trở nên dễ dàng hơn.

Về kiến trúc Multi-Agent, hệ thống áp dụng mô hình Supervisor với Supervisor Agent đóng vai trò điều phối, tiếp nhận yêu cầu và phân loại đến các sub-agents

chuyên biệt bao gồm: OutlineAgent cho việc tạo dàn ý bài giảng, ContentAgent cho việc sinh nội dung chi tiết, MediaAgent cho việc tìm kiếm hình ảnh minh họa, QuizAgent cho việc sinh câu hỏi luyện tập, và MergeAgent cho việc tổng hợp kết quả. Hệ thống còn tích hợp cơ chế Human-In-The-Loop (HITL) cho phép người dùng duyệt và chỉnh sửa outline trước khi tiến hành sinh nội dung chi tiết.

Về đóng góp chính, đề tài này tự xây dựng bộ máy truy xuất từ mức cơ sở, thiết kế kiến trúc Adaptive RAG với nhiều chiến lược truy xuất khác nhau, xây dựng hệ thống Multi-Agent cho sinh slide bài giảng với các sub-agents chuyên biệt, và tích hợp cơ chế HITL trong quy trình sinh slide.

1.4 Bố cục đồ án

Phần còn lại của báo cáo đồ án tốt nghiệp này được tổ chức như sau.

Chương 2 trình bày phạm vi nghiên cứu, tổng quan bài toán, khảo sát các giải pháp liên quan và cơ sở lý thuyết cần thiết cho đề tài. Chương này làm rõ vì sao bài toán hỏi đáp trên SGK Tin học THPT cần một hệ thống RAG có khả năng kiểm soát phạm vi dữ liệu, truy xuất đúng ngữ cảnh và đánh giá riêng các thành phần trong pipeline.

Chương 3 trình bày phương pháp đề xuất. Nội dung chương bao gồm kiến trúc tổng thể, nền tảng tri thức, cơ chế truy hồi ngữ cảnh, tầng điều phối tác tử, tầng sinh nội dung đa tác tử và phương pháp đánh giá. Đây là chương mô tả trực tiếp cách hệ thống xử lý dữ liệu SGK, thực hiện retrieval, reranking, generation và tổ chức các tác vụ sinh học liệu.

Chương 4 trình bày quá trình cài đặt, kiểm thử và kết quả thực nghiệm. Chương này mô tả môi trường thực nghiệm, dữ liệu đánh giá, các kịch bản kiểm thử, kết quả retrieval trước và sau rerank, kết quả đánh giá generation, độ trễ hệ thống và các phân tích lỗi quan trọng.

Chương 5 trình bày các giải pháp và đóng góp nổi bật của đồ án. Các đóng góp chính gồm tổ chức nền tảng tri thức từ SGK, thiết kế RAG core có truy hồi lai và adaptive retrieval, xây dựng tầng điều phối yêu cầu học tập, triển khai workflow multi-agent cho tác vụ sinh nội dung và tích hợp các cơ chế kiểm soát chất lượng đầu ra.

Cuối cùng, Chương 6 tổng kết kết quả đạt được, nêu các hạn chế còn tồn tại và đề xuất hướng phát triển trong tương lai.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT, KHẢO SÁT VÀ ĐỀ XUẤT GIẢI PHÁP

Chương 1 đã trình bày bối cảnh, mục tiêu và định hướng tổng quát của đề tài. Trên cơ sở đó, chương này tập trung làm rõ phạm vi nghiên cứu, ngữ cảnh bài toán, các hướng giải pháp liên quan và những nền tảng kỹ thuật cần thiết để hiểu phương pháp được đề xuất ở chương sau. Trọng tâm của chương không phải là mô tả các chức năng phần mềm, mà là xây dựng lập luận nghiên cứu: vì sao bài toán hỏi đáp trên SGK Tin học THPT cần một hệ thống RAG có khả năng truy xuất theo phạm vi, xếp hạng lại ngữ cảnh và đánh giá riêng từng thành phần trong pipeline.

Nội dung chương được tổ chức thành bốn phần. Mục đầu tiên xác định phạm vi nghiên cứu và các giới hạn của đề án. Mục thứ hai trình bày tổng quan bài toán hỏi đáp giáo dục dựa trên tài liệu. Mục thứ ba khảo sát các nhóm giải pháp liên quan và chỉ ra khoảng trống còn tồn tại. Mục cuối cùng trình bày các kiến thức nền tảng và hướng tiếp cận mà đề án lựa chọn.

2.1 Phạm vi nghiên cứu

Đối tượng nghiên cứu của đề án là hệ thống hỏi đáp dựa trên tài liệu (document-grounded question answering) cho môn Tin học THPT. Hệ thống tiếp nhận câu hỏi tự nhiên của người dùng, truy xuất ngữ cảnh liên quan từ SGK, sau đó sử dụng LLM để sinh câu trả lời dựa trên phần ngữ cảnh đã được truy xuất. Vì vậy, bài toán không được xem là một chatbot hội thoại mở thông thường, mà là một hệ thống hỏi đáp có ràng buộc nguồn tri thức rõ ràng.

Phạm vi dữ liệu của đề tài gồm SGK Tin học lớp 10, 11 và 12 thuộc hai bộ sách Cánh Diều (CD) và Kết Nối Tri Thức (KNTT). Đây là nguồn dữ liệu có cấu trúc phân cấp theo bộ sách, lớp, chủ đề, bài học và mục nội dung. Cấu trúc này có ý nghĩa quan trọng đối với bài toán retrieval, vì nhiều khái niệm Tin học có thể xuất hiện ở nhiều lớp hoặc nhiều bộ sách với mức độ trình bày khác nhau. Nếu hệ thống chỉ tìm kiếm theo toàn bộ kho dữ liệu mà không xét phạm vi, câu trả lời có thể lấy nhầm nội dung thuộc lớp hoặc bộ sách khác.

Đối tượng sử dụng chính của hệ thống gồm học sinh, giáo viên và người tự học môn Tin học THPT. Với học sinh, hệ thống cần hỗ trợ đặt câu hỏi tự nhiên, giải thích khái niệm, ôn tập nội dung bài học và định hướng luyện tập. Với giáo viên, hệ thống có thể hỗ trợ chuẩn bị học liệu như dàn ý slide, giáo án hoặc câu hỏi kiểm tra. Tuy nhiên, các tác vụ sinh học liệu này đều phụ thuộc vào một nền tảng chung là khả năng truy xuất đúng ngữ cảnh từ SGK trước khi sinh nội dung.

Trọng tâm nghiên cứu của đề án nằm ở pipeline retrieval và generation trong

kiến trúc Retrieval-Augmented Generation (RAG). Thành phần retrieval được đánh giá định lượng thông qua benchmark truy hồi, trong đó một câu hỏi kiểm thử gắn với một nguồn đúng kỳ vọng. Thành phần generation được đánh giá theo hướng chất lượng đầu ra và mức độ bám nguồn, ưu tiên rubric thủ công hoặc kết quả đánh giá đã có sẵn thay vì mở rộng chi phí đánh giá tự động bằng LLM. Các thành phần như giao diện, API, quản lý phiên, triển khai hạ tầng và đóng gói phần mềm được trình bày ở mức cần thiết để chứng minh hệ thống có thể vận hành và kiểm thử, nhưng không phải trọng tâm nghiên cứu chính.

Trong phạm vi đề án, các tác vụ phụ như sinh slide, soạn giáo án hoặc tạo câu hỏi luyện tập được xem là các luồng ứng dụng trên cùng nền tảng RAG và multi-agent. Chúng có vai trò chứng minh khả năng mở rộng của kiến trúc, nhưng phần đánh giá định lượng sâu vẫn tập trung vào truy xuất ngữ cảnh và sinh câu trả lời bám tài liệu. Các hướng mở rộng như graph retrieval hoặc tối ưu DevOps không được xem là đóng góp chính nếu chưa có thực nghiệm đầy đủ.

Từ phạm vi trên, câu hỏi nghiên cứu chính của đề án có thể phát biểu như sau: làm thế nào để truy xuất đúng ngữ cảnh từ SGK Tin học THPT và sinh câu trả lời bám tài liệu, đồng thời đánh giá được chất lượng của từng bước trong pipeline RAG. Câu hỏi này dẫn trực tiếp đến các quyết định thiết kế ở Chương 3, đặc biệt là chunking theo cấu trúc SGK, truy hồi lai giữa keyword search và vector search, reranking, adaptive retrieval và benchmark retrieval theo single-gold.

2.2 Tổng quan về bài toán

2.2.1 Bài toán chatbot giáo dục

Việc ứng dụng AI trong giáo dục tạo ra nhu cầu về các hệ thống có khả năng tương tác tự nhiên với người học. Thay vì chỉ tra cứu tài liệu theo từ khóa, người học có xu hướng đặt câu hỏi bằng ngôn ngữ tự nhiên, yêu cầu giải thích lại khái niệm, đưa ví dụ hoặc hỗ trợ ôn tập theo một bài học cụ thể. Với môn Tin học THPT, nhu cầu này càng rõ ràng vì nội dung học bao gồm cả lý thuyết, thao tác thực hành, thuật ngữ chuyên ngành, quy trình xử lý dữ liệu và các khái niệm liên quan đến lập trình.

Một chatbot giáo dục dùng cho SGK cần đáp ứng đồng thời yêu cầu về ngôn ngữ tự nhiên và yêu cầu về tính chuẩn xác theo tài liệu. Nếu chỉ ưu tiên khả năng hội thoại, hệ thống có thể tạo câu trả lời mạch lạc nhưng không đúng phạm vi bài học. Ngược lại, nếu chỉ dựa vào tìm kiếm từ khóa, hệ thống khó xử lý các câu hỏi được diễn đạt lại hoặc có ngữ cảnh hội thoại. Do đó, bài toán đặt ra không chỉ là tạo một giao diện hỏi đáp, mà là thiết kế một pipeline có khả năng truy xuất nguồn phù hợp và sinh câu trả lời có căn cứ.

Dữ liệu SGK Tin học THPT có một số đặc điểm khác với tài liệu văn bản thông thường. Nội dung được tổ chức theo lớp, bộ sách, chủ đề, bài học và mục nhỏ. Một số chủ đề như dữ liệu, thuật toán, mạng máy tính hoặc đạo đức số có thể xuất hiện ở nhiều lớp nhưng với độ sâu khác nhau. Ngoài ra, mỗi bài học thường kết hợp nhiều loại nội dung như khái niệm, ví dụ, hoạt động, bài tập, ghi nhớ hoặc câu hỏi vận dụng. Những đặc điểm này khiến hệ thống cần nhận biết vai trò của từng đoạn tài liệu thay vì xem toàn bộ văn bản như một tập đoạn rời rạc không có cấu trúc.

2.2.2 Hệ thống hỏi đáp dựa trên tài liệu

Hệ thống hỏi đáp có thể được chia thành hai nhóm lớn: closed-book QA và open-book QA. Trong closed-book QA, mô hình trả lời chủ yếu dựa trên tri thức đã được học trong quá trình huấn luyện. Cách tiếp cận này phù hợp với tri thức phổ quát, nhưng không bảo đảm chính xác khi câu hỏi phụ thuộc vào một tài liệu cụ thể. Trong open-book QA hoặc document-grounded QA, hệ thống được cung cấp thêm tài liệu nguồn và câu trả lời phải dựa trên tài liệu đó.

Đề án này thuộc nhóm document-grounded QA. Khi người dùng hỏi một khái niệm trong SGK Tin học, hệ thống không nên chỉ dựa vào tri thức tham số của LLM, vì tri thức này có thể không trùng với chương trình giáo dục Việt Nam hoặc có thể vượt khỏi phạm vi lớp học. Thay vào đó, hệ thống cần tìm các chunk liên quan trong SGK, đưa các chunk này vào context, rồi yêu cầu LLM sinh câu trả lời dựa trên context được cung cấp.

Luồng xử lý tổng quát của một hệ thống hỏi đáp dựa trên tài liệu gồm tiếp nhận câu hỏi, phân tích ý định hoặc phạm vi, truy xuất tài liệu liên quan, xây dựng context, sinh câu trả lời và trả kết quả cho người dùng. Trong luồng này, retrieval là bước có ảnh hưởng trực tiếp đến chất lượng đầu ra. Nếu context bị thiếu hoặc sai nguồn, LLM có thể sinh câu trả lời không đầy đủ hoặc không đúng dù prompt được thiết kế tốt. Vì vậy, đánh giá riêng retrieval là yêu cầu quan trọng khi nghiên cứu một hệ thống RAG.

2.2.3 Thách thức đặc thù của dữ liệu SGK Tin học

Thách thức đầu tiên đến từ cấu trúc phân cấp của SGK. Cùng một thuật ngữ có thể xuất hiện trong nhiều bài hoặc nhiều lớp, nhưng ý nghĩa sự phạm và mức độ chi tiết không giống nhau. Ví dụ, một khái niệm liên quan đến dữ liệu có thể được trình bày ở lớp 10 như kiến thức nền, nhưng ở lớp 11 hoặc 12 lại gắn với bài toán xử lý dữ liệu hoặc cơ sở dữ liệu. Vì vậy, hệ thống cần lưu và sử dụng metadata về bộ sách, lớp, chủ đề và bài học trong quá trình retrieval.

Thách thức thứ hai là cách học sinh đặt câu hỏi thường không trùng với câu chữ trong SGK. Người học có thể hỏi bằng ngôn ngữ đời thường, dùng từ đồng nghĩa,

hỏi tiếp theo một lượt hội thoại trước đó hoặc bỏ qua thông tin về bài học đang xét. Điều này làm cho keyword search đơn thuần chưa đủ. Hệ thống cần kết hợp vector search, query rewriting hoặc cơ chế phân tích ngữ cảnh để tăng khả năng tìm đúng tài liệu.

Thách thức thứ ba là tính đa dạng của nội dung Tin học. Các câu hỏi về định nghĩa, quy trình, so sánh, ví dụ và vận dụng cần các loại context khác nhau. Câu hỏi định nghĩa thường cần đoạn mô tả khái niệm trực tiếp, trong khi câu hỏi quy trình có thể cần nhiều bước nằm trong cùng một mục. Nếu chunking làm mất mạch nội dung hoặc retrieval trả về các đoạn rời rạc, câu trả lời cuối cùng có thể thiếu tính sư phạm.

Những thách thức trên cho thấy yêu cầu cốt lõi của bài toán không chỉ là tích hợp LLM vào hệ thống học tập. Hệ thống cần một nền tảng tri thức được chuẩn hóa, cơ chế retrieval có xét phạm vi, bước reranking để cải thiện thứ tự candidate, và quy trình đánh giá có thể xác định lỗi đến từ retrieval hay generation.

2.3 Khảo sát các giải pháp liên quan

2.3.1 Các nhóm giải pháp hiện có

Các giải pháp liên quan đến bài toán trợ lý học tập có thể được chia thành một số nhóm chính. Nhóm thứ nhất là các chatbot rule-based hoặc hệ thống intent classification truyền thống. Các hệ thống này sử dụng luật, kịch bản hoặc bộ intent được định nghĩa trước để tạo phản hồi. Ưu điểm của nhóm này là dễ kiểm soát trong phạm vi nhỏ và ít xảy ra hallucination nếu câu hỏi nằm đúng kịch bản. Tuy nhiên, chúng kém linh hoạt khi người dùng diễn đạt khác với mẫu dự kiến, đồng thời chi phí mở rộng luật tăng nhanh khi số bài học và số dạng câu hỏi tăng lên.

Nhóm thứ hai là các hệ thống FAQ hoặc keyword search. Cách tiếp cận này phù hợp với các kho câu hỏi cố định hoặc tài liệu có từ khóa rõ ràng. Trong bối cảnh SGK, keyword search có thể xử lý tốt các câu hỏi chứa thuật ngữ chính xác, tên khái niệm hoặc cụm từ gần giống tài liệu. Hạn chế của nhóm này là khả năng hiểu câu hỏi tự nhiên còn thấp, đặc biệt khi học sinh diễn đạt lại bằng ngôn ngữ đời thường hoặc hỏi theo ngữ cảnh hội thoại.

Nhóm thứ ba là semantic search dựa trên embedding. Thay vì chỉ so khớp từ khóa, semantic search biểu diễn câu hỏi và đoạn tài liệu thành vector để tìm các đoạn gần nhau về mặt ngữ nghĩa. Cách tiếp cận này phù hợp với câu hỏi được diễn đạt khác với văn bản gốc. Tuy nhiên, nếu chỉ dùng vector search đơn lẻ, hệ thống vẫn có thể bỏ sót các thuật ngữ hiếm, tên lệnh hoặc ký hiệu đặc thù của Tin học. Ngoài ra, semantic search không tự giải quyết vấn đề sinh câu trả lời cuối cùng.

Nhóm thứ tư là các chatbot dùng LLM trực tiếp, chẳng hạn các nền tảng AI đa mục đích như ChatGPT, Gemini hoặc Claude. Ưu điểm của nhóm này là khả năng hội thoại tự nhiên, diễn giải linh hoạt và xử lý tốt nhiều yêu cầu phức tạp. Tuy nhiên, khi áp dụng trực tiếp vào SGK Tin học Việt Nam, các mô hình này thiếu ràng buộc về nguồn tài liệu, dễ trả lời vượt phạm vi chương trình hoặc sinh thông tin không có căn cứ trong SGK.

Nhóm thứ năm là các hệ thống RAG chatbot. RAG kết hợp retrieval với generation, cho phép hệ thống lấy tài liệu liên quan trước khi gọi LLM. Đây là hướng phù hợp hơn với dữ liệu SGK vì tri thức có thể được cập nhật bằng cách thay đổi kho tài liệu thay vì huấn luyện lại mô hình. Dù vậy, hiệu quả của RAG phụ thuộc mạnh vào chất lượng chunking, embedding, retrieval, reranking và cách xây dựng context.

Nhóm thứ sáu là hybrid hoặc adaptive RAG. Nhóm này mở rộng RAG cơ bản bằng cách kết hợp nhiều chiến lược tìm kiếm, lọc metadata theo phạm vi, viết lại truy vấn, truy xuất nhiều truy vấn và reranking. Cách tiếp cận này phù hợp với dữ liệu SGK có cấu trúc phân cấp, nhưng đi kèm độ phức tạp cao hơn về thiết kế, log và đánh giá.

2.3.2 Bảng so sánh các nhóm giải pháp

Để làm rõ mức độ phù hợp của từng nhóm giải pháp đối với bài toán chatbot giáo dục dựa trên SGK, Bảng 2.1 đối chiếu các nhóm giải pháp theo năm tiêu chí chính.

Nhóm giải pháp	Hiểu câu hỏi tự nhiên	Dùng dữ liệu riêng	Giảm hallucination	Phù hợp với SGK
Rule-based	Thấp	Trung bình	Cao nếu luật đủ	Thấp
FAQ/keyword search	Thấp–Trung bình	Có	Trung bình	Trung bình
Semantic search	Trung bình	Có	Phụ thuộc tài liệu trả về	Trung bình–Cao
LLM trực tiếp	Cao	Thấp nếu không nạp context	Thấp	Thấp–Trung bình
RAG	Cao	Cao	Trung bình–Cao	Cao
Hybrid/adaptive RAG	Cao	Cao	Cao hơn nếu retrieval tốt	Cao

Bảng 2.1: So sánh các nhóm giải pháp cho bài toán chatbot giáo dục dựa trên SGK

Bảng 2.1 cho thấy các nhóm giải pháp đơn giản như rule-based hoặc FAQ có ưu

điểm về khả năng kiểm soát trong phạm vi hẹp, nhưng khó mở rộng khi số lượng bài học và cách đặt câu hỏi tăng lên. LLM trực tiếp có khả năng xử lý ngôn ngữ tốt hơn, nhưng thiếu cơ chế gắn câu trả lời với dữ liệu SGK. RAG và hybrid/adaptive RAG phù hợp hơn với bài toán của đề án vì vừa tận dụng được năng lực sinh ngôn ngữ của LLM, vừa có thể kiểm soát nguồn tri thức thông qua retrieval.

2.3.3 Nhận xét và định vị giải pháp đề án

Từ khảo sát trên, có thể rút ra ba nhận xét chính. Thứ nhất, các hệ thống dựa trên luật hoặc FAQ chưa đáp ứng tốt nhu cầu hỏi đáp tự nhiên của học sinh. Người học thường không đặt câu hỏi theo đúng mẫu cố định, do đó hệ thống cần khả năng hiểu ngữ nghĩa và xử lý nhiều cách diễn đạt khác nhau. Thứ hai, LLM trực tiếp tuy mạnh về ngôn ngữ nhưng không bảo đảm câu trả lời bám SGK. Với giáo dục phổ thông, một câu trả lời nghe hợp lý nhưng vượt phạm vi hoặc sai nguồn có thể gây tác động tiêu cực đến quá trình học. Thứ ba, RAG là hướng tiếp cận phù hợp, nhưng cần được thiết kế và đánh giá cẩn thận vì lỗi retrieval có thể lan truyền sang bước generation.

Khoảng trống mà đề án hướng tới nằm ở giao điểm giữa ba yêu cầu: dữ liệu SGK tiếng Việt có cấu trúc phân cấp, câu hỏi tự nhiên từ người dùng, và nhu cầu đánh giá riêng từng node trong pipeline. Nhiều hệ thống hiện có chỉ đánh giá câu trả lời cuối cùng hoặc tập trung vào trải nghiệm sản phẩm, trong khi chưa phân tích rõ chất lượng của retrieval, reranking và generation. Đề án vì vậy định vị giải pháp theo hướng hybrid/adaptive RAG, trong đó hệ thống có thể truy xuất theo phạm vi sách, lớp, chủ đề, xếp hạng lại candidate và xây dựng benchmark riêng để đo chất lượng retrieval.

Đối với nhóm giải pháp thương mại đa mục đích, điểm khác biệt của đề án không nằm ở việc xây dựng một LLM mới, mà nằm ở cách tổ chức tri thức SGK và kiểm soát context đưa vào LLM. Đối với nhóm chatbot dịch vụ trong nước, điểm khác biệt nằm ở mục tiêu học thuật: hệ thống cần giải thích, truy xuất bài học, sinh học liệu và giữ ngữ cảnh sư phạm. Đối với các hệ thống nghiên cứu hoặc đề án đi trước, điểm khác biệt nằm ở việc tách retrieval và generation thành các thành phần có thể quan sát, kiểm thử và đánh giá riêng.

2.4 Cơ sở lý thuyết và hướng tiếp cận

2.4.1 Mô hình ngôn ngữ lớn và hạn chế khi dùng trực tiếp

LLM là mô hình học máy có khả năng sinh văn bản tự nhiên dựa trên ngữ cảnh đầu vào. Trong bài toán giáo dục, LLM có thể diễn giải khái niệm, tóm tắt nội dung, chuyển đổi yêu cầu thành câu trả lời có cấu trúc và hỗ trợ sinh học liệu. Tuy nhiên, năng lực ngôn ngữ của LLM không đồng nghĩa với việc mô hình luôn có

tri thức đúng theo SGK đang xét. Khi không được cung cấp context từ tài liệu, mô hình có thể trả lời dựa trên tri thức tham số đã học, trong đó có thể chứa thông tin ngoài chương trình, thông tin không cập nhật hoặc thông tin không phù hợp với cách diễn đạt của SGK Việt Nam.

Hạn chế này đặc biệt quan trọng trong môi trường giáo dục. Một câu trả lời sai hoặc vượt cấp không chỉ là lỗi kỹ thuật, mà còn có thể làm người học hiểu sai kiến thức. Ngoài ra, context window của LLM có giới hạn, nên không thể đưa toàn bộ SGK vào prompt cho mọi lượt hỏi. Vì vậy, đề án không dùng LLM như nguồn tri thức duy nhất, mà đặt LLM sau lớp retrieval để mô hình sinh câu trả lời dựa trên các chunk đã được truy xuất.

2.4.2 Embedding và tìm kiếm ngữ nghĩa

Embedding là kỹ thuật biểu diễn văn bản thành vector số trong không gian nhiều chiều. Khi hai đoạn văn có nội dung gần nhau, vector của chúng thường có độ tương đồng cao. Trong hệ thống hỏi đáp dựa trên SGK, embedding cho phép đưa câu hỏi của người dùng và các chunk tài liệu vào cùng một không gian biểu diễn, từ đó thực hiện vector search để tìm các đoạn có ý nghĩa gần với câu hỏi.

Tìm kiếm ngữ nghĩa có ưu điểm ở các truy vấn diễn đạt lại, dùng từ đồng nghĩa hoặc không trùng bề mặt với tài liệu. Tuy nhiên, vector search cũng có hạn chế khi gặp thuật ngữ hiếm, tên hàm, ký hiệu hoặc cụm từ kỹ thuật cần khớp chính xác. Vì lý do đó, hướng tiếp cận của đề án không chỉ dựa vào embedding đơn lẻ, mà kết hợp thêm keyword search, metadata filtering và reranking. Các công thức cụ thể cho BM25, cosine similarity và RRF được trình bày trong Chương 3 khi mô tả cơ chế truy hồi ngữ cảnh.

2.4.3 Retrieval-Augmented Generation

Retrieval-Augmented Generation là kiến trúc kết hợp giữa retrieval và generation. Trong pha chuẩn bị dữ liệu, tài liệu nguồn được làm sạch, chia thành chunk, tạo embedding và lưu vào chỉ mục kèm metadata. Trong pha truy vấn, hệ thống nhận câu hỏi, truy xuất các chunk liên quan, có thể xếp hạng lại candidate, sau đó đưa context vào prompt để LLM sinh câu trả lời.

RAG phù hợp với bài toán SGK vì tri thức của hệ thống được quản lý thông qua kho tài liệu thay vì phụ thuộc hoàn toàn vào mô hình. Khi bổ sung hoặc cập nhật tài liệu, hệ thống có thể tái lập chỉ mục mà không cần huấn luyện lại LLM. Ngoài ra, các chunk được truy xuất có thể được ghi log, hiển thị hoặc dùng cho phân tích lỗi, giúp quá trình đánh giá minh bạch hơn so với việc chỉ quan sát câu trả lời cuối cùng.

Tuy nhiên, RAG không tự bảo đảm chất lượng đầu ra. Nếu tài liệu được chunking không hợp lý, metadata sai, retriever lấy nhầm bài hoặc context đưa vào prompt chứa nhiều nhiễu, LLM vẫn có thể sinh câu trả lời không đạt yêu cầu. Vì vậy, phương pháp của đồ án xem retrieval là thành phần cần đánh giá định lượng riêng, còn generation được đánh giá theo mức độ đúng, đủ, bám context và rõ ràng.

2.4.4 Adaptive retrieval, reranking và query rewriting

Một truy vấn người dùng có thể thiếu thông tin về phạm vi, phụ thuộc vào hội thoại trước đó hoặc dùng cách diễn đạt không trùng với tài liệu. Query rewriting giúp chuẩn hóa hoặc làm rõ truy vấn trước khi retrieval. Trong khi đó, adaptive retrieval chọn chiến lược truy xuất tùy theo loại câu hỏi, scope và tín hiệu ngữ cảnh, thay vì dùng một cấu hình cố định cho mọi trường hợp.

Reranking là bước đánh giá lại danh sách candidate sau truy xuất ban đầu. Retriever thường ưu tiên tốc độ và độ phủ, nên top-k ban đầu có thể chứa các chunk chưa thật sự phù hợp. Reranker, thường được triển khai bằng cross-encoder, nhận cặp query–document và ước lượng lại mức độ liên quan. Cách làm này có thể cải thiện thứ tự context đưa vào LLM, nhưng cũng làm tăng latency và chi phí tính toán. Vì vậy, trong Chương 4, kết quả thực nghiệm cần so sánh trước và sau rerank để xác định lợi ích thực tế của bước này.

Trong đồ án, adaptive retrieval, query rewriting và reranking được xem là các cơ chế hỗ trợ cho RAG core. Chúng không thay thế mục tiêu chính là truy xuất đúng context, mà giúp hệ thống xử lý tốt hơn các câu hỏi mơ hồ, câu hỏi có scope cụ thể và câu hỏi được diễn đạt khác với văn bản SGK.

2.4.5 Multi-intent và điều phối yêu cầu học tập

Người dùng của hệ thống không chỉ đặt câu hỏi kiến thức, mà còn có thể yêu cầu tạo quiz, sinh slide, soạn giáo án hoặc kết hợp nhiều yêu cầu trong cùng một lượt hỏi. Vì vậy, hệ thống cần một lớp điều phối để phân tích intent, xác định task type, lập kế hoạch hành động và gọi đúng service. Lớp điều phối này giúp biến yêu cầu tự nhiên thành các hành động có cấu trúc, nhưng không phải trọng tâm đánh giá định lượng của đồ án.

Vai trò của multi-intent routing trong nghiên cứu này là bảo đảm request được đưa vào đúng pipeline. Với câu hỏi kiến thức, hệ thống cần ưu tiên retrieval và answer generation. Với yêu cầu sinh học liệu, hệ thống cần sử dụng context từ SGK rồi chuyển sang workflow multi-agent để xây dựng dàn ý, nội dung, media hoặc câu hỏi đánh giá. Cách tổ chức này cho phép các tác vụ khác nhau dùng chung nền tảng tri thức, đồng thời giữ cho retrieval vẫn là thành phần trung tâm.

2.4.6 Đánh giá hệ thống RAG

Đánh giá hệ thống RAG cần tách hai câu hỏi: hệ thống có truy xuất đúng context không, và LLM có sinh câu trả lời tốt từ context đó không. Nếu chỉ đánh giá câu trả lời cuối cùng, rất khó xác định lỗi đến từ retriever hay generator. Một câu trả lời sai có thể do context sai, context thiếu, prompt chưa tốt hoặc LLM suy diễn quá mức. Ngược lại, một câu trả lời đúng không nhất thiết chứng minh retrieval tốt, vì LLM có thể dựa vào tri thức sẵn có để trả lời.

Đối với retrieval, đề án sử dụng benchmark có expected source rõ ràng để tính các chỉ số như Hit@k, Recall@k, Precision@k, MRR và nDCG@k. Các chỉ số này cho phép đo khả năng đưa đúng nguồn vào top-k, thứ hạng của nguồn đúng và mức độ tập trung của kết quả truy xuất. Đối với generation, hướng đánh giá phù hợp hơn là dùng rubric về độ đúng nội dung, độ đầy đủ, mức độ bám context, độ rõ ràng và khả năng tránh hallucination. Cách tách đánh giá này giúp quá trình phân tích lỗi có cơ sở hơn và phù hợp với mục tiêu nghiên cứu của đề án.

2.4.7 Khoảng trống và hướng tiếp cận của đề án

Từ các phân tích trên, có thể thấy khoảng trống chính của bài toán nằm ở việc xây dựng một hệ thống RAG cho dữ liệu SGK tiếng Việt có khả năng kiểm soát scope và đánh giá được từng thành phần. Các giải pháp hiện có hoặc quá cứng nhắc khi dựa vào luật và từ khóa, hoặc quá mở khi dùng LLM trực tiếp, hoặc chưa đánh giá rõ chất lượng retrieval trước khi kết luận về chất lượng câu trả lời.

Hướng tiếp cận của đề án là thiết kế một pipeline RAG cho SGK Tin học THPT, trong đó tài liệu được chunking theo cấu trúc bài học, mỗi chunk được gắn metadata phục vụ lọc scope, retrieval kết hợp keyword search và vector search, candidate được rerank trước khi đưa vào generation, và các bước xử lý được ghi log để phục vụ đánh giá. Trên nền tảng đó, hệ thống mở rộng sang các tác vụ sinh học liệu bằng lớp điều phối và workflow multi-agent, nhưng vẫn giữ nguyên nguyên tắc grounding trên nội dung SGK.

Phần tiếp theo của báo cáo sẽ trình bày chi tiết phương pháp đề xuất. Cụ thể, Chương 3 mô tả kiến trúc tổng thể, nền tảng tri thức, RAG core, tầng điều phối tác tử, tầng sinh nội dung đa tác tử và phương pháp đánh giá. Những nội dung nền tảng trong chương này là cơ sở để giải thích các quyết định thiết kế và thực nghiệm ở các chương sau.

2.5 Kết chương

Chương này đã xác định phạm vi nghiên cứu của đề án là hệ thống hỏi đáp dựa trên SGK Tin học THPT theo hướng document-grounded QA. Chương cũng đã

phân tích đặc thù của dữ liệu SGK, làm rõ các thách thức về scope, ngôn ngữ tự nhiên, thuật ngữ Tin học và vai trò của retrieval trong chất lượng câu trả lời.

Thông qua khảo sát các nhóm giải pháp liên quan, chương đã chỉ ra rằng RAG và hybrid/adaptive RAG là hướng tiếp cận phù hợp hơn so với rule-based, FAQ hoặc LLM trực tiếp khi yêu cầu câu trả lời phải bám sát SGK. Các kiến thức nền tảng về LLM, embedding, RAG, query rewriting, reranking, multi-intent và đánh giá hệ thống RAG tạo cơ sở cho Chương 3, nơi phương pháp đề xuất được trình bày chi tiết hơn về kiến trúc, các node xử lý và phương pháp đánh giá.

CHƯƠNG 3. THIẾT KẾ GIẢI PHÁP VÀ PHƯƠNG PHÁP ĐÁNH GIÁ

Chương 2 đã trình bày cơ sở lý thuyết, khảo sát các hướng tiếp cận và xác định Retrieval-Augmented Generation là hướng tiếp cận phù hợp cho bài toán hỏi đáp trên SGK Tin học THPT. Trên cơ sở đó, chương này mô tả phương pháp được đề xuất, bao gồm kiến trúc tổng thể, các thành phần xử lý chính và cách tổ chức đánh giá cho các node trọng tâm trong pipeline.

Nội dung chương được trình bày theo hướng nghiên cứu, tập trung vào dữ liệu, retrieval, reranking, generation và evaluation, không đi sâu vào chi tiết cài đặt phần mềm.

3.1 Tổng quan kiến trúc hệ thống

3.1.1 Bài toán và yêu cầu

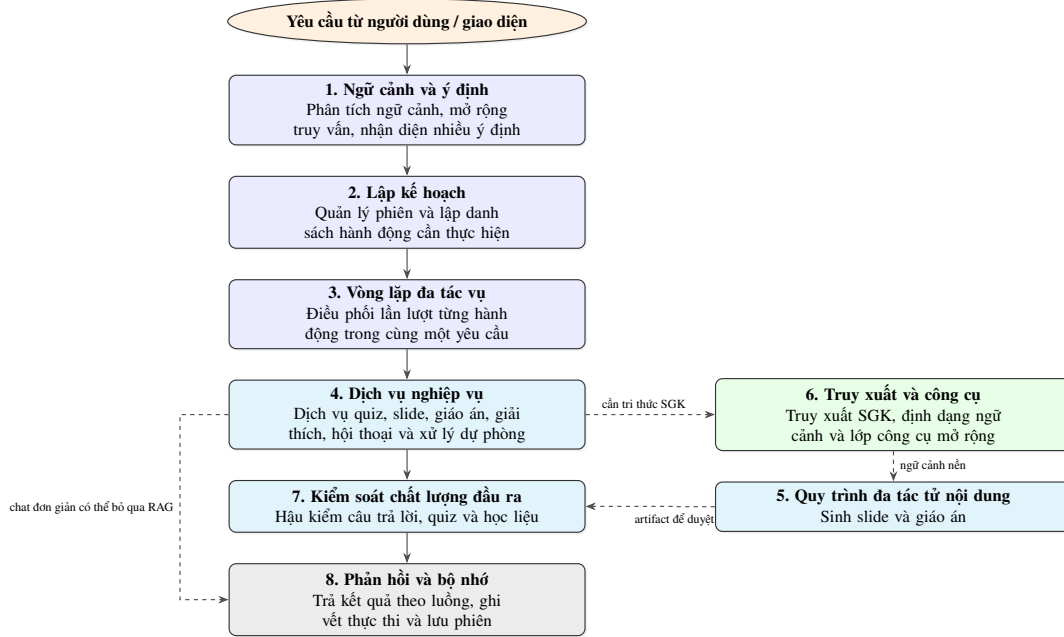
Bài toán của đề án là xây dựng một trợ lý giáo dục cho môn Tin học THPT, hỗ trợ người học và giáo viên khai thác nội dung từ sách giáo khoa trong môi trường tương tác số. Phạm vi dữ liệu của hệ thống được giới hạn vào sách giáo khoa Tin học lớp 10, 11 và 12 của hai bộ sách Cánh Diều và Kết Nối Tri Thức. Trên nền dữ liệu này, hệ thống không chỉ thực hiện tác vụ hỏi đáp kiến thức mà còn hỗ trợ các tác vụ sinh học liệu có cấu trúc, đòi hỏi khả năng tổng hợp và tổ chức lại nội dung.

Ba nhóm tác vụ chính của hệ thống gồm hỏi đáp dựa trên SGK, sinh slide bài giảng và soạn giáo án. Với tác vụ hỏi đáp, hệ thống cần truy xuất các đoạn ngữ cảnh liên quan thông qua *Retriever*, sau đó sử dụng *LLM* để sinh câu trả lời bám sát tài liệu nguồn. Với tác vụ sinh slide và giáo án, hệ thống cần bao phủ nội dung bài học ở mức tổng thể, đồng thời tổ chức lại tri thức theo trình tự sư phạm thay vì chỉ trả về các đoạn văn bản rời rạc. Do đó, bài toán không dừng ở một pipeline *Retrieval-Augmented Generation* (RAG) đơn giản, mà mở rộng thành bài toán điều phối nhiều bước xử lý với các yêu cầu khác nhau về truy xuất, tổng hợp và kiểm soát chất lượng.

Trên cơ sở đó, phương pháp đề xuất cần thỏa mãn một số yêu cầu chính. Thứ nhất, hệ thống phải khai thác tri thức từ SGK theo đúng phạm vi lớp, bộ sách và chủ đề nhằm hạn chế sai lệch ngữ cảnh. Thứ hai, cơ chế truy xuất phải xử lý được cả câu hỏi cụ thể lẫn các yêu cầu tổng hợp nội dung ở mức bài học. Thứ ba, phần sinh nội dung phải có cơ chế kiểm soát chất lượng đầu ra đối với các sản phẩm dài như slide và giáo án. Cuối cùng, kiến trúc hệ thống phải hỗ trợ đánh giá riêng chất lượng truy hồi của *Retriever/Reranker* và chất lượng sinh của *LLM*.

3.1.2 Kiến trúc tổng thể và luồng xử lý một yêu cầu

Kiến trúc của hệ thống được tổ chức theo hai tầng *Agent* đặt trên nền *Retrieval-Augmented Generation*. Tầng điều phối tác tử đảm nhiệm việc phân tích yêu cầu, lập kế hoạch và lựa chọn nhánh xử lý. Tầng sinh nội dung *Multi-Agent* đảm nhiệm việc tạo các sản phẩm dài như slide và giáo án. Hai tầng này cùng sử dụng một lõi truy xuất tri thức chung để bảo đảm đầu ra được grounding trên nội dung SGK.



Hình 3.1: Kiến trúc tổng thể của phương pháp đề xuất theo hướng phân tầng, trong đó lõi truy xuất tri thức là nền dùng chung cho các nhánh xử lý

Trong Hình 3.1, trục chính của vòng đời xử lý đi theo chuỗi từ yêu cầu đầu vào, phân tích ngữ cảnh và ý định, lập kế hoạch, điều phối đa tác vụ, thực thi dịch vụ nghiệp vụ, kiểm soát chất lượng đầu ra, rồi đến phản hồi và lưu trạng thái phiên. Cách biểu diễn này cho thấy bộ điều phối của hệ thống hoạt động theo một luồng xử lý chính, trong khi các năng lực truy xuất và sinh học liệu dài được gọi vào theo từng loại yêu cầu, thay vì xuất hiện như một bước tuần tự bắt buộc cho mọi truy vấn.

Ở bên phải của trục chính, khối “Truy xuất và công cụ” được đặt như một nền dùng chung cho các nhánh cần tri thức SGK. Từ khối “Dịch vụ nghiệp vụ”, hệ thống gọi xuống lõi truy xuất để lấy ngữ cảnh thông qua *Retriever*; nếu tác vụ là sinh slide hoặc giáo án thì ngữ cảnh này tiếp tục được đưa vào “Quy trình đa tác tử nội dung”. Bố cục này phản ánh quan hệ phụ thuộc giữa các thành phần: truy xuất tri thức diễn ra trước khi tầng *Multi-Agent* tổng hợp sản phẩm.

Phần cuối của sơ đồ cho thấy mọi đầu ra đều đi qua khối “Kiểm soát chất lượng đầu ra” trước khi trả về cho người dùng. Với câu trả lời ngắn, lớp này kiểm tra mức

độ phù hợp và bám sát ngữ cảnh; với quiz, slide và giáo án, lớp này thực hiện hậu kiểm trước khi công bố kết quả. Hai mũi tên nét đứt cũng thể hiện rằng chat đơn giản có thể bỏ qua lõi RAG, trong khi các tác vụ cần grounding trên SGK sẽ kết nối đến nền truy xuất chung.

3.1.3 Công nghệ nền

Về mặt công nghệ, phương pháp được hiện thực hóa trên một số thành phần nền tảng tương ứng với từng vai trò trong kiến trúc. Mô hình ngôn ngữ lớn Gemini được sử dụng như *LLM* cho các bước cần suy luận ngôn ngữ như nhận diện ý định, viết lại truy vấn và sinh nội dung. LangGraph được sử dụng cho bộ điều phối nội dung ở tầng sinh học liệu, nơi quy trình có nhiều bước trung gian và có thể tạm dừng để con người tham gia duyệt. Ở tầng truy xuất, Qdrant được sử dụng như một *Vector Database* để lưu trữ và tìm kiếm ngữ nghĩa trên tập chunk đã được lập chỉ mục.

Đối với biểu diễn tri thức, hệ thống sử dụng mô hình *Embedding* tiếng Việt để chuyển mỗi khối nội dung SGK thành vector ngữ nghĩa, qua đó bảo toàn tốt hơn đặc trưng ngôn ngữ của dữ liệu trong không gian biểu diễn. Song song với đó, một thành phần tìm kiếm từ khóa kiểu BM25 được dùng để bổ sung tín hiệu lexical, sau đó kết quả được hợp nhất bằng *Reciprocal Rank Fusion*. Ở giai đoạn sau truy xuất, mô hình *Reranker* dạng cross-encoder được dùng để sắp xếp lại các ứng viên theo mức độ liên quan đối với truy vấn.

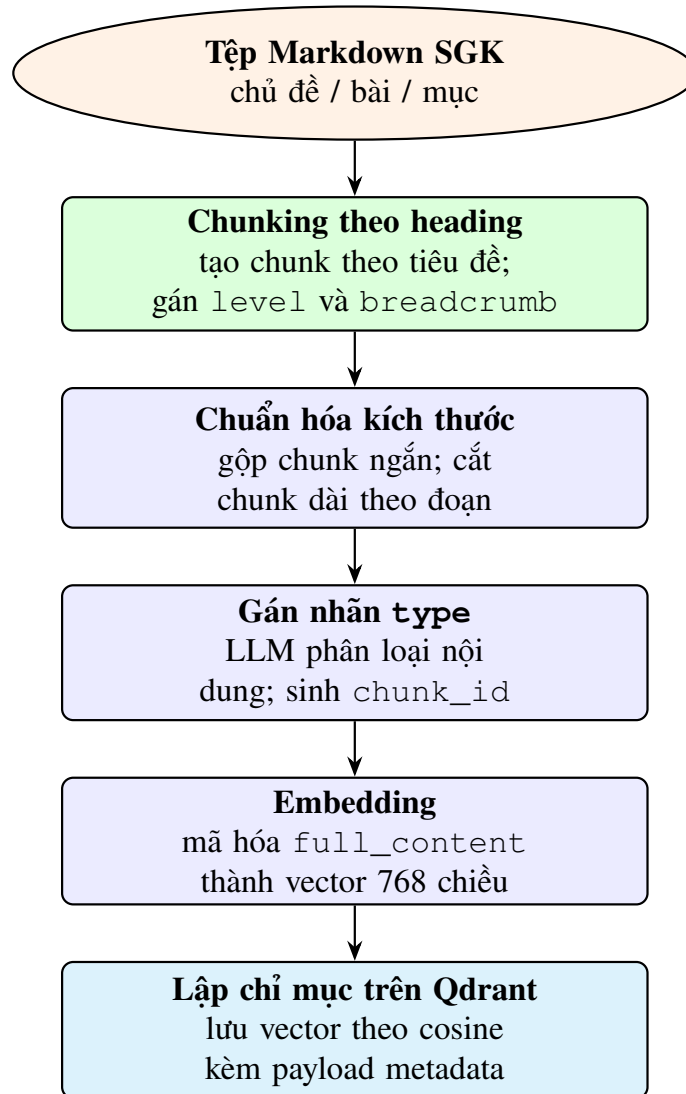
Trong phạm vi chương này, các công nghệ trên chỉ được trình bày ở mức cần thiết để làm rõ vai trò trong phương pháp đề xuất. Thông số cài đặt cụ thể của từng công nghệ sẽ được trình bày ở chương tiếp theo.

3.2 Nền tảng tri thức: tiền xử lý, chunking và lập chỉ mục

Mọi đầu ra của hệ thống đều được grounding trên nội dung sách giáo khoa, do đó chất lượng của tầng tri thức nền ảnh hưởng trực tiếp đến hiệu quả truy hồi và sinh nội dung. Phần này mô tả quy trình biến văn bản SGK thô thành một tập chunk có cấu trúc, được embedding và lập chỉ mục để phục vụ truy hồi. Hình 3.2 tóm tắt quy trình này, trong đó bước chunking theo tiêu đề (heading-based chunking) là thành phần trung tâm.

Về bản chất, tầng tri thức của hệ thống không chỉ là một kho lưu trữ dữ liệu tĩnh mà là kết quả của một pipeline tiền xử lý nhiều bước. Mỗi bước trong pipeline này tác động trực tiếp đến chất lượng truy hồi ở giai đoạn sau. Nếu ranh giới chunk không phù hợp, mô hình embedding sẽ biểu diễn sai đơn vị kiến thức; nếu metadata không đầy đủ, tầng truy hồi sẽ khó giới hạn đúng phạm vi lớp, bộ sách và bài học; nếu chỉ mục vector không đồng bộ với dữ liệu nguồn, kết quả truy xuất có thể mất tính ổn định giữa các lần chạy thực nghiệm. Vì vậy, việc mô tả rõ quy trình xây

dựng tầng tri thức là cần thiết trước khi phân tích cơ chế retrieval ở mục kế tiếp.



Hình 3.2: Quy trình từ văn bản SGK đến tập chunk được lập chỉ mục, với chunking theo tiêu đề là bước trung tâm

Hình 3.2 mô tả quy trình một chiều biến tập bài học SGK thành các chunk sẵn sàng cho truy hồi, gồm năm bước nối tiếp. Đầu vào là tập Markdown của một bài học, trong đó cấu trúc phân cấp (chủ đề, tiêu đề bài, tiêu đề mục) đã được thể hiện qua các mốc tiêu đề. Bước trung tâm là *chunking theo heading*: một máy trạng thái quét tuần tự văn bản và chốt một chunk mới tại mỗi mốc tiêu đề, đồng thời gán trường `level` (độ sâu tiêu đề) và dựng chuỗi breadcrumb cho chunk đó. Tiếp theo, bước chuẩn hóa kích thước gộp các chunk quá ngắn và cắt các chunk quá dài để đưa độ dài về khoảng hợp lý mà không phá vỡ ranh giới ngữ nghĩa. Bước thứ tư phân loại vai trò sự phạm (`type`) của mỗi chunk bằng một mô hình ngôn ngữ nhẹ và sinh `chunk_id` bất định. Cuối cùng, mỗi chunk được embedding cùng breadcrumb thành vector 768 chiều và được lập chỉ mục trên Qdrant kèm toàn bộ

payload metadata. Trong các bước này, chunking theo tiêu đề giữ vai trò quyết định vì nó định hình ranh giới và tính mạch lạc của mọi chunk ở các bước sau; ba mục tiếp theo lần lượt trình bày chi tiết nguồn dữ liệu và metadata, cơ chế chunking, và khâu vector hóa – lưu trữ.

3.2.1 Nguồn SGK, cấu trúc phân cấp và lược đồ metadata

Dữ liệu nền của hệ thống là sách giáo khoa Tin học THPT của hai bộ Cánh Diều và Kết Nối Tri Thức, trải đều ba khối lớp 10, 11 và 12. Toàn bộ nội dung được số hóa về định dạng Markdown và tổ chức theo đúng cấu trúc phân cấp tự nhiên của sách: mỗi bộ sách (*book*) chia theo khối lớp (*grade*), mỗi lớp gồm nhiều chủ đề (*topic*), mỗi chủ đề gồm nhiều bài học (*lesson*), và mỗi bài học lại được chia thành các mục theo cấp độ tiêu đề. Cấu trúc thư mục phản ánh trực tiếp phân cấp này, trong đó mỗi tệp Markdown tương ứng với một bài học và tệp `README.md` ở mỗi thư mục chủ đề lưu mã cùng tên chủ đề.

Sau bước xử lý, tập tri thức thu được gồm 2.348 chunk, bao phủ 36 chủ đề và 186 bài học khác nhau. Bảng 3.1 trình bày phân bố số lượng chunk theo bộ sách và khối lớp.

Bộ sách	Lớp 10	Lớp 11	Lớp 12	Tổng
Cánh Diều (CD)	401	439	364	1.204
Kết Nối Tri Thức (KNTT)	401	350	393	1.144
Tổng	802	789	757	2.348

Bảng 3.1: Phân bố số lượng chunk theo bộ sách và khối lớp

Số liệu trong Bảng 3.1 cho thấy dữ liệu tương đối cân bằng giữa hai bộ sách, với 1.204 chunk thuộc bộ Cánh Diều và 1.144 chunk thuộc bộ Kết Nối Tri Thức. Phân bố theo khối lớp cũng không có chênh lệch lớn, lần lượt là 802 chunk ở lớp 10, 789 chunk ở lớp 11 và 757 chunk ở lớp 12. Phân bố này giúp giảm nguy cơ tăng truy hồi thiên lệch về một bộ sách hay một khối lớp cụ thể.

Mỗi chunk được gắn một lược đồ metadata thống nhất nhằm phục vụ lọc phạm vi khi truy hồi và truy vết nguồn khi sinh nội dung. Bảng 3.2 mô tả các trường chính của lược đồ này.

Bảng 3.2 cho thấy mỗi chunk không chỉ lưu nội dung văn bản mà còn mang theo các trường metadata về phạm vi học liệu, vị trí phân cấp và nguồn gốc dữ liệu. Tập trường này là cơ sở để tăng truy hồi thực hiện lọc theo bộ sách, lớp, chủ đề và bài học, đồng thời hỗ trợ truy vết nguồn ở các bước sinh nội dung và đánh giá.

Trường `type` phân loại mỗi chunk theo vai trò sư phạm thay vì chỉ theo hình thức văn bản, gồm sáu nhãn: *theory* (lý thuyết, khái niệm, ví dụ), *exercise* (luyện

Trường	Kiểu	Ý nghĩa
chunk_id	uuid	Định danh duy nhất, sinh tất định
book	str	Bộ sách (CD / KNTT)
grade	str	Khối lớp (10 / 11 / 12)
topic, topic_name	str	Mã và tên chủ đề
lesson, lesson_name	str	Mã và tên bài học
section_title	str	Tiêu đề mục chứa chunk
level	int	Độ sâu tiêu đề (1–6)
type	str	Loại nội dung sư phạm
breadcrumb	str	Đường dẫn ngữ cảnh: chủ đề > bài > mục
content	str	Nội dung văn bản của chunk
full_content	str	Ghép breadcrumb và content để vector hóa
source_path	str	Đường dẫn tệp nguồn

Bảng 3.2: Lược đồ metadata của một chunk

tập, bài tập), *objective* (mục tiêu bài học), *application* (vận dụng), *summary* (tóm tắt) và *note* (lưu ý). Bảng 3.3 trình bày phân bố số lượng chunk theo từng nhãn.

Loại (type)	Vai trò	Số lượng	Tỷ lệ
<i>theory</i>	Lý thuyết, khái niệm, ví dụ	1.682	71,6%
<i>exercise</i>	Luyện tập, bài tập	250	10,6%
<i>objective</i>	Mục tiêu bài học	193	8,2%
<i>application</i>	Vận dụng	122	5,2%
<i>summary</i>	Tóm tắt	85	3,6%
<i>note</i>	Lưu ý	16	0,7%
Tổng		2.348	100%

Bảng 3.3: Phân bố số lượng chunk theo loại nội dung sư phạm (type)

Số liệu trong Bảng 3.3 cho thấy phần lớn nội dung thuộc nhãn *theory*, chiếm khoảng 71,6% tổng số chunk. Phân bố này phản ánh đặc thù của SGK, trong đó phần trình bày khái niệm, giải thích và ví dụ chiếm ưu thế so với các thành phần như bài tập, mục tiêu hay tóm tắt. Nhãn loại nội dung này tạo cơ sở cho việc khai thác chọn lọc ở các tác vụ phía sau, chẳng hạn ưu tiên chunk *objective* và *summary* khi dựng dàn ý bài giảng, hoặc chunk *exercise* khi sinh câu hỏi luyện tập.

3.2.2 Chunking theo tiêu đề

Khác với cách chia chunk theo cửa sổ ký tự cố định, chiến lược **chunking theo tiêu đề (heading-based chunking)** được sử dụng trong đồ án xác định ranh giới chunk trực tiếp từ cấu trúc tiêu đề Markdown của SGK. Theo đó, mỗi chunk tương ứng với một đơn vị nội dung mạch lạc về mặt sư phạm. Cụ thể, bộ chunking hoạt động như một máy trạng thái quét tuần tự từng dòng của tệp bài học; mỗi khi gặp một mốc phân cấp — dòng chủ đề (“CHỦ ĐỀ ...”), tiêu đề bài (“# Bài X: ...”) hay tiêu đề mục (“## ...”, “### ...”) — hệ thống chốt chunk đang tích lũy và mở một

chunk mới gắn với mốc đó. Cách chia này bảo đảm mỗi chunk nằm trọn trong một mục của một bài, không bị cắt ngang giữa câu hay giữa các ý.

Để làm rõ cơ chế này, có thể xét một tài liệu giả lập gồm ba tầng tiêu đề như sau:

```
CHU DE 2. Mang may tinh va Internet
```

```
# Bai 3. Giao tiep tren mang
```

```
Doan mo dau cua bai hoc.
```

```
## 1. Dia chi IP
```

```
Dia chi IP dung de dinh danh thiet bi tren mang.
```

```
### Vi du
```

```
May tinh A co dia chi 192.168.1.10.
```

```
## 2. Ten mien
```

```
Ten mien giup nguoi dung de nho dia chi tai nguyen hon.
```

Với đầu vào trên, hệ thống không tạo một chunk duy nhất cho toàn bộ bài học mà chốt chunk tại từng mốc tiêu đề. Bảng 3.4 minh họa kết quả phân đoạn tương ứng.

Chunk giả lập	Level	Breadcrumb	Nội dung chính
c_1	1	Chủ đề 2 > Bài 3	Đoạn mở đầu của bài học
c_2	2	Chủ đề 2 > Bài 3 > 1. Địa chỉ IP	Khái niệm địa chỉ IP
c_3	3	Chủ đề 2 > Bài 3 > 1. Địa chỉ IP > Ví dụ	Ví dụ về địa chỉ IP cụ thể
c_4	2	Chủ đề 2 > Bài 3 > 2. Tên miền	Khái niệm tên miền

Bảng 3.4: Ví dụ minh họa cách chunking theo tiêu đề trên một tài liệu ba tầng heading

Bảng 3.4 cho thấy ranh giới chunk được quyết định bởi tiêu đề gần nhất đang có hiệu lực. Đoạn mở đầu ngay sau tiêu đề bài được gán vào chunk mức 1. Khi gặp mục “1. Địa chỉ IP”, hệ thống tạo một chunk mới ở mức 2. Phần “Ví dụ” bên trong mục này tiếp tục được tách thành một chunk mức 3, do nó được mở bởi một tiêu đề con riêng. Tương tự, khi gặp mục “2. Tên miền”, hệ thống chốt chunk trước đó và mở ra một chunk mới. Nhờ cách chia này, mỗi chunk giữ được cả nội dung trực tiếp lẫn vị trí phân cấp trong bài học.

Mỗi chunk được gán trường `level` bằng đúng độ sâu tiêu đề trong Markdown: tiêu đề bài tương ứng `level = 1`, các mục chính `level = 2`, các mục

con `level` ≥ 3 . Phân bố thực tế cho thấy nội dung tập trung chủ yếu ở `level 2` (1.229 chunk) và `level 3` (921 chunk). Song song, mỗi chunk mang một chuỗi *breadcrumb* dạng “tên chủ đề > tên bài > tiêu đề mục” làm ngữ cảnh phân cấp gọn nhẹ; breadcrumb được ghép vào trước nội dung khi embedding và khi xếp hạng lại bằng *Reranker*, qua đó hỗ trợ phân biệt các chunk có câu chữ tương tự nhưng thuộc bài khác nhau.

Vì ranh giới chunk bám theo tiêu đề nên kích thước chunk phụ thuộc độ dài tự nhiên của từng mục. Để xử lý hai trường hợp cực đoan, một bước chuẩn hóa kích thước nhẹ được áp dụng sau khi chia: các chunk quá ngắn (dưới 100 ký tự) được gộp vào chunk liền trước nếu cùng bài và cùng loại nội dung, còn các chunk quá dài (trên 2.000 ký tự) được cắt theo ranh giới đoạn văn. Các ngưỡng này chỉ tác động tới một phần nhỏ tập chunk và không làm thay đổi bản chất chia theo tiêu đề; độ dài chunk thực tế dao động quanh trung bình 651 ký tự. Cuối cùng, nhãn `type` được tinh chỉnh bằng một mô hình ngôn ngữ nhẹ (Gemini 2.5 Flash-Lite) trên cơ sở nhãn heuristic ban đầu, có cơ chế đệm (cache) để bảo đảm tính tái lập; đồng thời mỗi chunk nhận một `chunk_id` sinh tất định bằng hàm băm trên (bộ sách, lớp, chủ đề, bài, breadcrumb, thứ tự, dấu vân nội dung), giúp việc tái dựng tập chunk có tính idempotent.

3.2.3 Biểu diễn vector tiếng Việt và lưu trữ Qdrant

Để hỗ trợ truy hồi ngữ nghĩa, mỗi chunk được chuyển thành một vector số thực bằng mô hình embedding tiếng Việt `dangvantuan/vietnamese-document-embedding`, sinh vector 768 chiều. Văn bản đưa vào mô hình là trường `full_content` (breadcrumb ghép với nội dung) thay vì chỉ riêng nội dung, để ngữ cảnh phân cấp được phản ánh trong không gian vector. Sau khi mã hóa, các vector được chuẩn hóa L_2 nhằm bảo đảm độ đo cosine quy về tích vô hướng; quá trình này có kiểm tra và xử lý các vector suy biến (chuẩn gần không hoặc giá trị không hữu hạn) để duy trì tính ổn định của tập chỉ mục.

Tập vector cùng metadata được lưu vào Qdrant dưới một collection tên `educational_chatbot`, cấu hình không gian 768 chiều với độ đo khoảng cách cosine. Mỗi điểm (point) trong Qdrant nhận `chunk_id` làm định danh, mang vector ngữ nghĩa và toàn bộ payload metadata ở Bảng 3.2; nhờ đó hệ thống có thể vừa tìm kiếm theo độ tương đồng vừa lọc cứng theo bộ sách, lớp, chủ đề trong cùng một truy vấn. Quá trình nạp dữ liệu được thực hiện theo lô và có kiểm tra tính nhất quán giữa số chunk và số vector. Ngoài ra, một dấu vân (fingerprint) trên nguồn embedding được lưu kèm để phát hiện khi dữ liệu chunk thay đổi và chỉ tính lại vector khi cần. Thiết kế lưu trữ này cho phép tăng truy hồi nạp dữ liệu từ kho Qdrant hoặc từ

tập vector cục bộ trong các kịch bản triển khai và thực nghiệm.

3.3 Cơ chế truy hồi ngữ cảnh (RAG core)

3.3.1 Truy hồi lai BM25 và dense, hợp nhất bằng Reciprocal Rank Fusion

Lỗi truy hồi của hệ thống được xây dựng theo hướng truy hồi lai, kết hợp giữa tìm kiếm từ khóa (*keyword search*) và tìm kiếm ngữ nghĩa theo vector (*vector search*). Trong đó, *keyword search* phù hợp với các truy vấn có mức trùng khớp bề mặt cao với SGK, còn *vector search* phù hợp với các truy vấn diễn đạt lại hoặc dùng từ đồng nghĩa.

Với *keyword search*, hệ thống sử dụng BM25. Xuất phát từ TF-IDF, trong đó tần suất từ được ký hiệu bởi Công thức 3.1:

$$TF(t, d) = f(t, d), \quad (3.1)$$

và tần suất nghịch đảo tài liệu được tính theo Công thức 3.2:

$$IDF(t) = \log \frac{N - n(t) + 0.5}{n(t) + 0.5}, \quad (3.2)$$

Trong đó:

- t là một term trong truy vấn.
- N là số tài liệu trong tập chỉ mục.
- $n(t)$ là số tài liệu chứa từ t .

Trên cơ sở đó, điểm BM25 của truy vấn q đối với chunk d được tính như sau:

$$BM25(q, d) = \sum_{t \in q} IDF(t) \cdot \frac{f(t, d) (k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \cdot \frac{|d|}{avgdl}\right)}. \quad (3.3)$$

Trong đó:

- q là truy vấn.
- t là một term thuộc truy vấn q .
- $f(t, d)$ là số lần xuất hiện của từ t trong chunk d .
- $|d|$ là độ dài của chunk d .
- $avgdl$ là độ dài chunk trung bình của toàn bộ tập chỉ mục.

- k_1 và b là các siêu tham số điều chỉnh ảnh hưởng của tần suất từ và độ dài tài liệu.

Với *vector search*, mỗi truy vấn q và mỗi chunk d được ánh xạ thành embedding $\mathbf{e}_q$ và $\mathbf{e}_d$. Độ tương đồng được tính bằng cosine similarity theo Công thức 3.4:

$$\text{cos_sim}(\mathbf{e}_q, \mathbf{e}_d) = \frac{\mathbf{e}_q \cdot \mathbf{e}_d}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_d\|_2}. \quad (3.4)$$

Trong đó:

- $\mathbf{e}_q$ là embedding của truy vấn.
- $\mathbf{e}_d$ là embedding của chunk.
- $\mathbf{e}_q \cdot \mathbf{e}_d$ là tích vô hướng giữa hai vector.
- $\|\cdot\|_2$ là chuẩn Euclid.

Do các vector được chuẩn hóa L_2 trước khi lập chỉ mục, độ đo này quy về so sánh hướng trong không gian vector và được dùng trực tiếp trong truy vấn Qdrant.

Hai danh sách kết quả từ BM25 và *vector search* được hợp nhất bằng *Reciprocal Rank Fusion* (RRF), thay vì trộn trực tiếp điểm số. Với một chunk d , điểm RRF được xác định theo Công thức 3.5:

$$\text{RRF}(d) = \sum_{r \in \mathcal{R}} \frac{1}{k + \text{rank}_r(d)}, \quad (3.5)$$

Trong đó:

- $\mathcal{R}$ là tập các danh sách xếp hạng thành phần.
- $\text{rank}_r(d)$ là thứ hạng của chunk d trong danh sách r .
- k là hằng số làm tròn.

Cách hợp nhất này tránh phụ thuộc vào khác biệt thang điểm giữa BM25 và cosine similarity, đồng thời giữ lại các chunk có thứ hạng cao ở ít nhất một kênh truy hồi. Kết quả của bước này là một tập candidate ban đầu để chuyển sang bước reranking.

3.3.2 Xếp hạng lại bằng cross-encoder

Sau bước truy hồi lai, hệ thống áp dụng một bước *reranking* bằng cross-encoder để tái xếp hạng tập candidate. Mục tiêu của bước này là thay thế thứ tự dựa trên mức tương đồng sơ cấp bằng thứ tự gần hơn với mức độ liên quan thực sự giữa truy vấn và chunk.

Reranker nhận đầu vào là cặp (*query*, *context*) và sinh ra một điểm số liên quan

$$s_{\text{rerank}} = f_{\theta}(q, d), \quad (3.6)$$

Trong đó:

- q là truy vấn.
- d là chunk ứng viên.
- θ là tập tham số của mô hình.
- f_{θ} là hàm chấm điểm của cross-encoder.

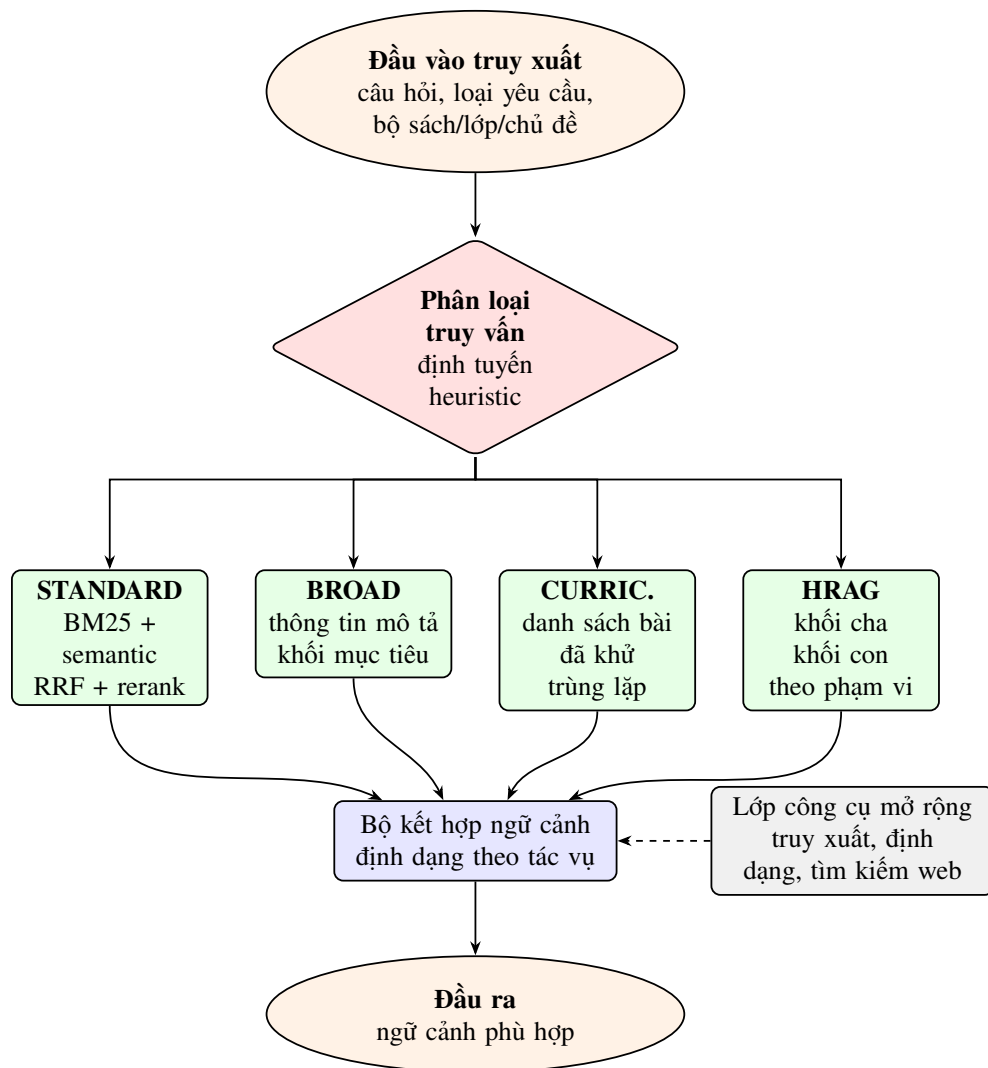
Khác với *vector search*, mô hình này mã hóa đồng thời cả truy vấn và chunk, do đó có thể khai thác tương tác token-level giữa hai vế. Trong triển khai hiện tại, phần văn bản đưa vào *Reranker* không chỉ gồm *content* mà còn ghép thêm *breadcrumb*, nhằm cung cấp thêm ngữ cảnh phân cấp của chunk trong SGK.

Sau khi tính lại điểm số, hệ thống chọn top- N chunk để xây dựng ngữ cảnh cuối. Giá trị N được đặt theo loại tác vụ: với hỏi đáp ngắn, N nhỏ hơn để hạn chế nhiễu; với slide hoặc giáo án, N lớn hơn để tăng độ bao phủ. Vì vậy, reranking vừa là bước cải thiện thứ hạng, vừa là bước chọn lọc ngữ cảnh đầu vào cho tầng generation.

3.3.3 Truy hồi thích ứng (Adaptive RAG)

Không phải mọi truy vấn trong hệ thống đều cần cùng một cách truy hồi. Câu hỏi định nghĩa ngắn, yêu cầu liệt kê bài học, truy vấn tổng quan theo chủ đề và yêu cầu sinh slide từ một bài học có phạm vi ngữ cảnh khác nhau. Vì vậy, hệ thống sử dụng cơ chế *Adaptive RAG* để chọn strategy truy hồi phù hợp theo loại truy vấn và thông tin phạm vi hiện có.

Về mặt lý thuyết, cơ chế này có hai thành phần chính. Thành phần thứ nhất là bước phân loại truy vấn, nhằm xác định loại nhu cầu truy hồi trước khi chọn strategy. Thành phần thứ hai là bước ràng buộc phạm vi (*scope filtering*) theo bộ sách, lớp hoặc chủ đề nếu các metadata này đã được nhận diện ở tầng điều phối. Nhờ đó, không gian tìm kiếm được thu hẹp trước khi thực hiện BM25 hoặc *vector search*. Trong trường hợp strategy đang dùng không trả về đủ ngữ cảnh, hệ thống có thể fallback về cấu hình mặc định để bảo đảm độ phủ.



Hình 3.3: Cơ chế truy hồi thích ứng của hệ thống với bộ phân loại truy vấn và các chiến lược truy xuất tương ứng

Trong Hình 3.3, bộ phân loại truy vấn được đặt ở trung tâm vì đây là thành phần quyết định strategy truy hồi sẽ được áp dụng trước. Bốn strategy ở tầng dưới tương ứng với bốn kiểu nhu cầu truy hồi khác nhau, trong khi khối “Bộ kết hợp ngữ cảnh” chuẩn hóa đầu ra của từng strategy về một cấu trúc chung cho tầng trên sử dụng. Trong cấu trúc này, *STANDARD* là strategy mặc định gồm BM25, *vector search*, RRF và reranking. *BROAD* ưu tiên các chunk mô tả ở mức khái quát. *CURRICULUM* phục vụ các truy vấn liệt kê bài học hoặc cấu trúc chương trình, trong đó đầu ra cần ở mức danh sách bài thay vì đoạn nội dung chi tiết. *HRAG* kết hợp các chunk mức cao và mức chi tiết trong cùng phạm vi bài học. Sau khi truy hồi, khối “Bộ kết hợp ngữ cảnh” thực hiện chuẩn hóa đầu ra để các strategy khác nhau có thể cung cấp ngữ cảnh theo cùng một cấu trúc cho tầng trên sử dụng.

3.4 Tầng điều phối tác tử

3.4.1 Sự cần thiết của tầng điều phối tác tử

Nếu hệ thống chỉ phục vụ một tác vụ hỏi đáp đơn giản, một pipeline tuyến tính gồm query analysis, retrieval và answer generation là đủ. Tuy nhiên, bài toán của đồ án bao gồm đồng thời nhiều nhóm yêu cầu khác nhau, từ hỏi đáp khái niệm, giải thích bài học, sinh slide đến soạn giáo án. Các yêu cầu này khác nhau không chỉ ở output format mà còn ở retrieval scope, mức độ phụ thuộc vào multi-turn context và số bước xử lý trung gian. Do đó, việc dùng một pipeline cố định cho mọi truy vấn làm giảm khả năng thích ứng của hệ thống.

Tầng điều phối tác tử được đưa vào để tách riêng ba trách nhiệm: intent understanding, execution routing và action execution. Trong kiến trúc này, bộ điều phối không được thiết kế như một *Agent* tự trị bao trùm toàn bộ hệ thống. Thay vào đó, nó là một lớp orchestration mỏng, giữ *request state*, chỉ gọi *LLM* ở các vị trí cần suy luận ngôn ngữ như intent routing hoặc query rewriting, và dùng deterministic logic cho planning, dispatch và session management. Cách tổ chức này giúp giảm số lượt gọi mô hình, hạn chế latency và làm rõ ranh giới giữa reasoning dựa trên ngôn ngữ với quyết định có thể mô hình hóa bằng rule.

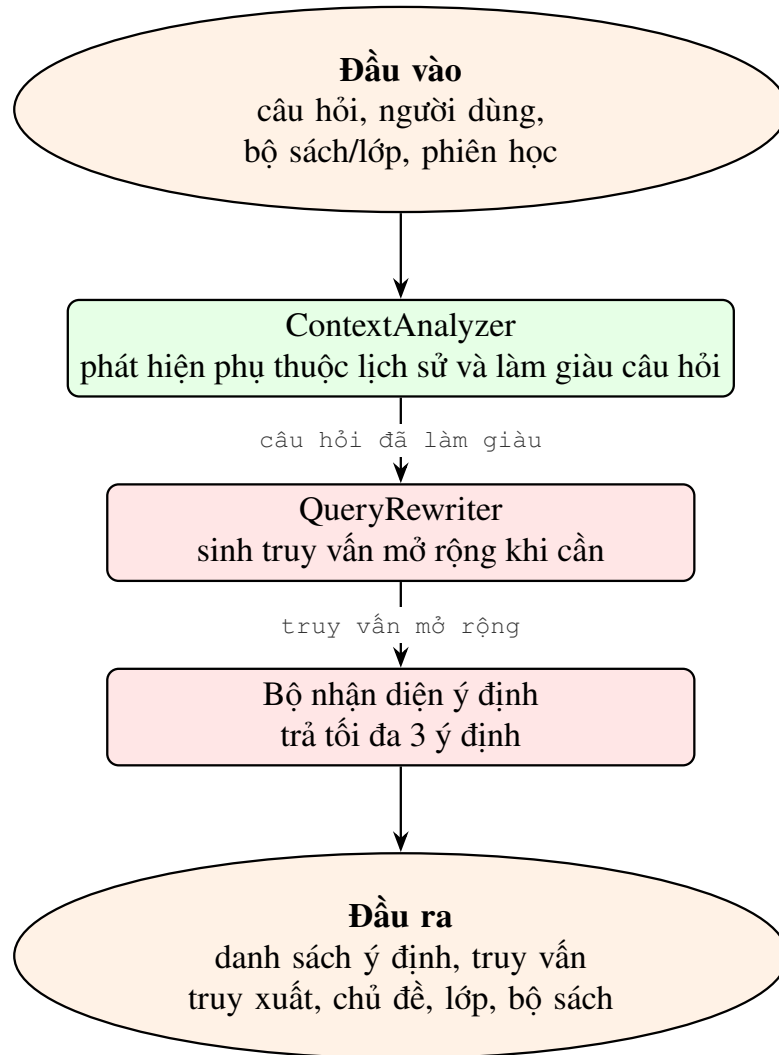
Vai trò thứ hai của tầng điều phối là hỗ trợ multi-intent request. Trong thực tế sử dụng, một câu hỏi có thể đồng thời chứa nhiều action, chẳng hạn vừa yêu cầu giải thích khái niệm, vừa yêu cầu tạo bài tập hoặc sinh dàn ý bài giảng. Khi đó, hệ thống cần phân rã đầu vào thành một *action list* có thứ tự, tránh xử lý trùng lặp và duy trì được *session state* xuyên suốt phiên làm việc. Đây là lý do phần điều phối được mô hình hóa thành một tầng riêng, thay vì gắn trực tiếp vào từng dịch vụ nghiệp vụ.

3.4.2 Định tuyến ý định bằng mô hình ngôn ngữ lớn

Điểm vào đầu tiên của tầng điều phối là bài toán intent routing. Với cùng một surface query, hệ thống cần xác định truy vấn đang hướng tới hỏi đáp kiến thức, sinh học liệu dài, hay một thao tác phụ trợ khác như thống kê tiến độ hoặc ôn câu sai. Ngoài intent type, hệ thống còn cần trích xuất các scope field như bộ sách, lớp, chủ đề và bài học khi thông tin này xuất hiện trong câu hỏi hoặc có thể suy ra từ session context.

Trong phương pháp đề xuất, bước intent routing được giao cho *LLM* thay vì heuristic thuần túy. Lý do là cùng một intent có thể được diễn đạt theo nhiều cách khác nhau, đặc biệt trong ngôn ngữ học sinh. Router cần xử lý được indirect query, composite query và cả các trường hợp câu hỏi ngắn nhưng phụ thuộc mạnh vào turn history. Đầu ra của bước này không phải một nhãn duy nhất, mà là một danh

sách tối đa ba intent có thứ tự ưu tiên, đi kèm các scope field phục vụ các tầng sau.



Hình 3.4: Khối phân tích ngữ cảnh và định tuyến ý định trong tầng điều phối

Hình 3.4 mô tả processing chain từ câu hỏi gốc đến đầu ra intent routing. Đầu vào ban đầu gồm câu hỏi hiện tại, thông tin người dùng, bộ sách hoặc lớp được chọn trên giao diện và *session state*. Thành phần *ContextAnalyzer* đứng trước cùng vì đây là bước quyết định câu hỏi hiện tại có phụ thuộc vào turn history hay không. Nếu phụ thuộc, hệ thống lấy các lượt hội thoại gần và liên quan để mở rộng câu hỏi hiện tại trước khi chuyển sang bước sau.

Sau bước phân tích ngữ cảnh, hệ thống chỉ gọi *QueryRewriter* khi truy vấn cần được reformulate để hỗ trợ *retrieval*. Thành phần này không thay thế câu hỏi gốc, mà tạo ra một hoặc nhiều expanded query nhằm tăng recall của tầng truy hồi trong các trường hợp câu hỏi quá ngắn, thiếu chủ ngữ hoặc dùng diễn đạt không trùng với SGK. Cuối cùng, bộ định tuyến ý định dùng *LLM* để phân tích yêu cầu đã được làm giàu và trả về danh sách intent, retrieval query và các scope field như chủ đề, lớp và bộ sách. Đầu ra này là input trực tiếp cho tầng planning.

3.4.3 Làm giàu ngữ cảnh hội thoại và viết lại truy vấn

Context enrichment là bước cần thiết vì nhiều truy vấn trong thực tế không tự mang đủ thông tin để tăng truy hồi hoạt động chính xác. Các câu như “giải thích kỹ hơn”, “bài này khác gì bài trước” hoặc “làm cho em vài câu hỏi” chỉ có thể được hiểu đúng nếu hệ thống nối chúng với turn trước đó. Do vậy, trước khi query rewriting, hệ thống trước hết phải xác định liệu câu hỏi hiện tại có phụ thuộc vào history hay không.

Trong hệ thống, *ContextAnalyzer* được hiện thực bằng deterministic logic. Thành phần này quan sát câu hỏi hiện tại, *session state* và một số surface cue như đại từ hồi chỉ, câu rút gọn hoặc việc người dùng tiếp tục một chủ đề đã mở trước đó. Khi phát hiện context dependency, bộ phân tích trích xuất một cửa sổ hội thoại ngắn, ưu tiên các turn gần nhất và các turn có cùng lesson scope. Kết quả của bước này là một phiên bản câu hỏi đã được disambiguate, đủ để tăng dưới xử lý mà không cần mang theo toàn bộ history.

QueryRewriter đóng vai trò chuyển câu hỏi đã làm giàu thành dạng phù hợp hơn cho *retrieval*. Với truy vấn hỏi đáp, bộ viết lại có thể sinh thêm các paraphrase gần nghĩa để tăng recall của *keyword search* và *vector search*. Với truy vấn sinh học liệu dài, bộ viết lại ưu tiên làm rõ lesson scope, topic scope và output type. Việc tách bước này khỏi *IntentRouter* giúp hệ thống kiểm soát hai bài toán khác nhau: một bài toán là intent understanding, bài toán còn lại là retrieval-oriented reformulation.

Trong thiết kế hiện tại, conversation context sau khi được phân tích không được đưa thẳng vào toàn bộ pipeline ở mọi bước. Thay vào đó, nó được nén thành một structured representation gồm câu hỏi đã làm rõ, expanded query, scope field và cờ đánh dấu mức độ phụ thuộc history. Cách chuẩn hóa này giúp các thành phần planning và retrieval dùng lại cùng một input schema, thay vì mỗi thành phần tự diễn giải lịch sử theo cách riêng.

3.4.4 Quản lý trạng thái hội thoại và vòng đời session

State management là nền của tầng điều phối, không phải một feature tách rời. Sau bước intent routing, hệ thống phải quyết định câu hỏi hiện tại có thuộc cùng một phiên làm việc với các turn trước đó hay không, rồi mới có thể lập kế hoạch hành động chính xác. Vì vậy, trong phương pháp đề xuất, *SessionManager* được đặt đúng giữa intent routing và action planning, đóng vai trò duy trì continuity của multi-turn context, đồng thời ngăn việc trộn lẫn các workflow khác bản chất trong cùng một session.

Mô hình trạng thái được tổ chức theo ba lớp. Ở lớp chung nhất là *Session*, lưu định danh phiên, người dùng, topic hiện tại, intent chính, book scope, timestamp và lịch sử message. Trên nền đó, hệ thống chỉ kích hoạt một trong hai intent-specific state là *QuizState* hoặc *SlideState*, tương ứng với hai nhóm workflow có cấu trúc trạng thái khác nhau. *QuizState* phục vụ các vòng sinh câu hỏi, chấm đáp án và ôn tập; *SlideState* phục vụ các workflow sinh slide, giáo án và các điểm dừng *human-in-the-loop*. Thiết kế này giúp session vừa giữ được conversation history dùng chung, vừa cô lập được state nghiệp vụ đặc thù của từng task type.

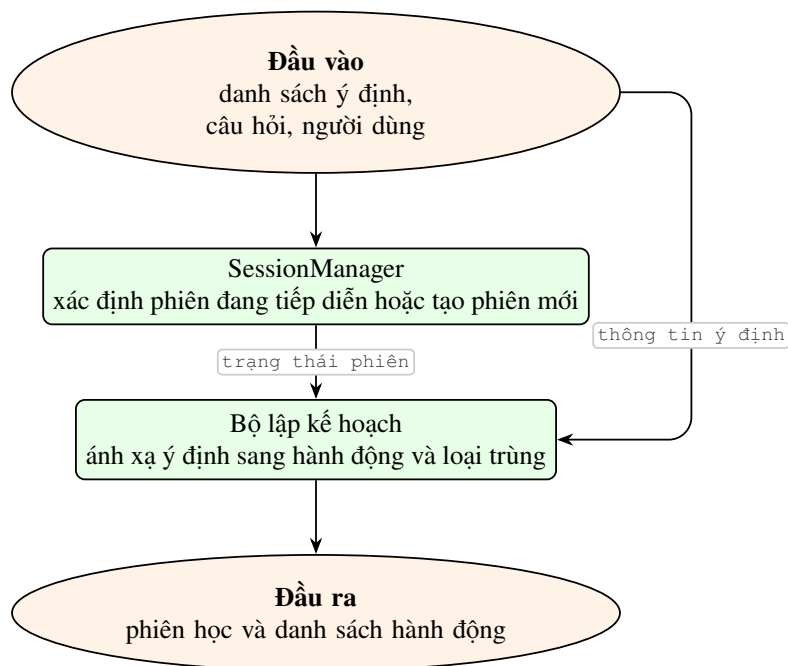
Luật vòng đời session được thực hiện bởi *SessionManager.resolve_session()*. Nếu người dùng chưa có session hiện hành, hệ thống tạo session mới. Nếu *IntentRouter* phát hiện đổi topic rõ ràng, session cũ được lưu lại và phiên mới được khởi tạo để tránh kéo theo context không còn liên quan. Với các yêu cầu sinh nội dung, hệ thống còn xét thêm *task type transition*. Chẳng hạn, một session đang ở nhánh quiz có thể tiếp tục cho nhiều round cùng loại, nhưng khi chuyển từ quiz sang slide hoặc từ slide sang quiz thì hệ thống tách phiên mới, vì hai workflow này có state object, artifact và logic điều phối khác nhau. Khi không có tín hiệu đổi topic hoặc đổi task type, session hiện tại được reuse để bảo toàn ngữ cảnh hội thoại.

Vai trò quan trọng thứ hai của session management là cung cấp history theo đúng mức chi tiết cho các bước suy luận ngôn ngữ. Trong thiết kế hiện tại, *Session.get_context_messages()* trả về một cửa sổ message gần nhất để inject vào intent detection, giúp router phân biệt các truy vấn mơ hồ như “làm tiếp”, “chấm câu này” hay “giải thích ý trên”. Ở nhánh contextualization, *ContextAnalyzer* và *QueryRewriter* không dùng toàn bộ session thô, mà chỉ lấy phần history liên quan để tạo *context snippet* rồi viết lại truy vấn phục vụ *retrieval*. Cách dùng này giữ cho session trở thành nguồn trạng thái thống nhất của tầng điều phối, nhưng vẫn kiểm soát được độ dài context và ranh giới thông tin được truyền sang từng node.

Việc tách state management thành một cơ chế thiết kế riêng trong tầng điều phối mang lại hai lợi ích. Thứ nhất, action planner luôn nhận được một *session state* đã được chuẩn hóa về topic, task type và history context, nhờ đó quyết định tiếp theo không phải suy diễn lại trạng thái từ đầu. Thứ hai, các workflow multi-turn như “sinh quiz rồi chấm đáp án”, “sinh dàn ý rồi duyệt outline”, hoặc “hỏi tiếp trong cùng bài học” có thể vận hành liên tục mà không làm rò rỉ ngữ cảnh sang một tác vụ khác. Chi tiết cài đặt của lớp lưu phiên, cơ chế auto-save và đồng bộ truy cập sẽ được trình bày ở Chương 4, vì chúng thuộc phạm vi hiện thực hóa hơn là phạm vi thiết kế phương pháp.

3.4.5 Lập kế hoạch hành động và thực thi đa hành động

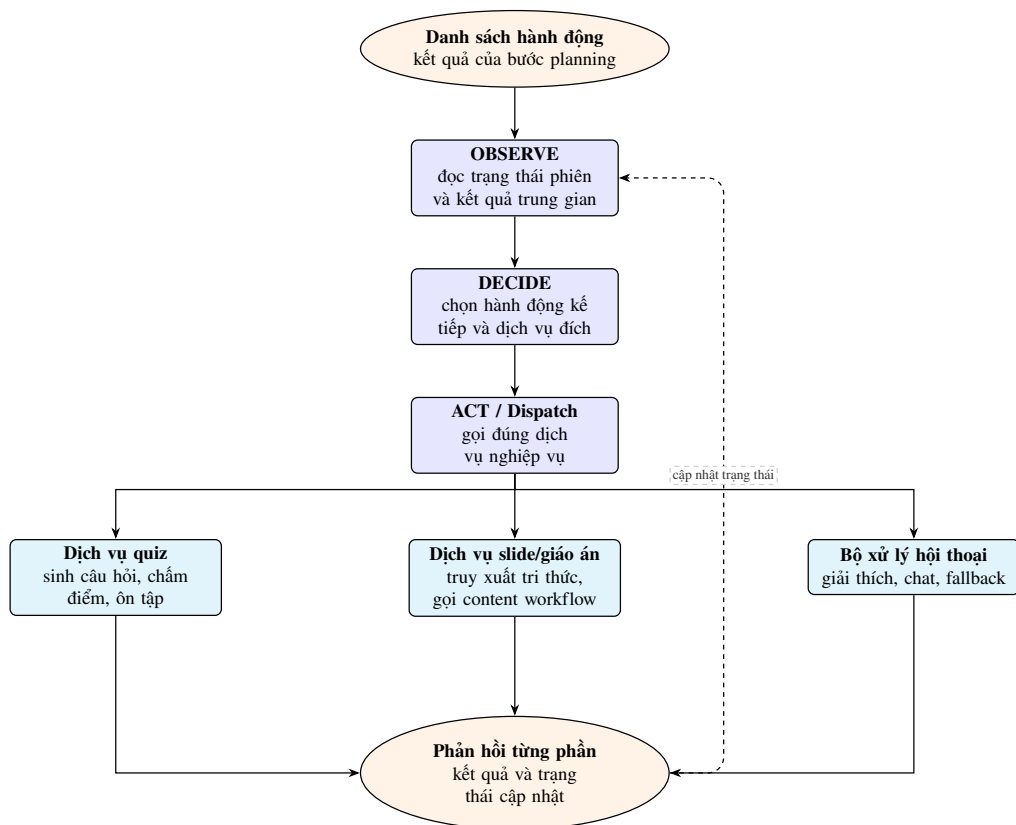
Sau khi có danh sách intent, hệ thống chuyển sang bước action planning. Mục tiêu của bước này là ánh xạ từng intent sang một business action cụ thể, đồng thời quyết định execution order khi trong cùng một yêu cầu xuất hiện nhiều intent. Khác với bước intent routing, bài toán planning ở đây được giải quyết bằng deterministic logic thay vì *LLM*. Lý do là action space của hệ thống là hữu hạn và đã biết trước, bao gồm hỏi đáp, giải thích khái niệm, sinh slide, sinh giáo án, sinh quiz, chấm đáp án, ôn câu sai và một số thao tác hỗ trợ khác.



Hình 3.5: Cơ chế lập kế hoạch hành động ở tầng điều phối

Hình 3.5 cho thấy bước planning gồm hai khối chính. Trước hết, *SessionManager* xác định session hiện tại của người dùng, từ đó quyết định nên tiếp tục workflow đang có hay khởi tạo phiên mới. Tiếp theo, planner nhận đồng thời thông tin intent và *session state* để sinh ra một *action list* đã được deduplicate. Chẳng hạn, nếu nhiều intent liên tiếp cùng dẫn tới một hành động hỏi đáp trong cùng một lesson scope, hệ thống không tạo nhiều bước xử lý trùng nhau mà gộp về một action duy nhất.

Sau bước planning, *action list* được thực thi theo một execution loop đa hành động. Về bản chất, đây là một chu trình *OBSERVE–DECIDE–ACT*: hệ thống đọc *session state* và intermediate result, chọn action kế tiếp cần chạy, rồi dispatch sang đúng business service tương ứng. Cấu trúc này cho phép hệ thống stream dần kết quả của từng action, đồng thời cập nhật intermediate state mà không cần chờ toàn bộ composite request kết thúc mới sinh phản hồi.



Hình 3.6: Vòng thực thi đa hành động trong tầng điều phối theo chu trình quan sát – quyết định – hành động

Hình 3.6 làm rõ execution loop sau bước planning. Khối *OBSERVE* đọc *session state* và các intermediate result đã có. Khối *DECIDE* chọn action kế tiếp cần chạy dựa trên *action list* còn lại. Khối *ACT* tương ứng với execution dispatcher, chịu trách nhiệm route sang đúng business service. Với các action gắn với quiz, luồng được chuyển sang dịch vụ quiz. Với các action cần sinh học liệu dài, luồng được chuyển sang dịch vụ slide/giáo án để tiếp tục gọi nhánh *Multi-Agent*. Với các action giải thích hoặc hội thoại ngắn, hệ thống dùng chat handler chuyên biệt. Kết quả sau mỗi vòng được ghi trở lại vào *session state* để phục vụ iteration kế tiếp.

Thiết kế này cho phép bộ điều phối giữ vai trò trung tâm của phương pháp đề xuất mà không làm nó trở thành một monolithic reasoning block. *LLM* chỉ xuất hiện ở những bước cần language understanding hoặc query reformulation; còn state management, planning và action dispatch vẫn được thực hiện bằng deterministic mechanism. Nhờ đó, hệ thống duy trì được tính giải thích được của processing flow, đồng thời vẫn đủ linh hoạt để hỗ trợ composite request trong cùng một session.

3.5 Tầng sinh nội dung đa tác tử

3.5.1 Mô hình Supervisor–Specialist và lý do lựa chọn

Đối với các tác vụ sinh slide và soạn giáo án, hệ thống không chỉ cần tạo văn bản đầu ra mà còn phải bảo đảm đồng thời nhiều ràng buộc như cấu trúc sư phạm, độ bám nguồn, khả năng gắn học liệu minh họa, mức độ đầy đủ của phần luyện tập và tính nhất quán của toàn bộ artifact cuối. Nếu toàn bộ quá trình này được giao cho một *single-agent* duy nhất, prompt sẽ phải bao hàm cùng lúc yêu cầu lập dàn ý, viết nội dung, đề xuất media, sinh đánh giá và tự kiểm tra chất lượng. Cách tiếp cận đó làm tăng tải nhận thức của mô hình, khiến ranh giới trách nhiệm mờ đi và khó truy vết khi đầu ra không đạt yêu cầu.

Vì vậy, đồ án lựa chọn kiến trúc *Supervisor–Specialist*. Trong kiến trúc này, một *Content Supervisor* giữ vai trò điều phối workflow, còn từng *specialist agent* chỉ xử lý một lớp nhiệm vụ hẹp đã được xác định trước. Supervisor không trực tiếp sinh nội dung bài học mà thực hiện ba chức năng chính: (i) phân rã yêu cầu thành các bước con, (ii) ràng buộc thứ tự thực thi giữa các bước, và (iii) thu thập kết quả trung gian để quyết định bước tiếp theo. Cách phân tách này giúp workflow sinh nội dung gắn với một quy trình xử lý có trạng thái (*stateful workflow*) hơn là một lần gọi mô hình đơn lẻ.

Một ưu điểm quan trọng khác của mô hình này là khả năng kiểm soát *grounding*. Trong pipeline hiện tại, phần outline được sinh trước để xác lập khung sư phạm, sau đó agent viết nội dung mới được phép khai thác các chunk đã truy hồi để điền nội dung chi tiết cho từng slide hoặc từng mục giáo án. Việc tách outline và drafting thành hai bước độc lập giúp giảm nguy cơ mô hình vừa tự quyết cấu trúc vừa tự mở rộng nội dung ngoài phạm vi nguồn. Đồng thời, các bước tùy chọn như gợi ý media hoặc sinh đánh giá nhúng cũng có thể được thêm vào hoặc bỏ qua mà không làm thay đổi hợp đồng dữ liệu của các bước cốt lõi.

Xét về mặt hiện thực hệ thống, kiến trúc *Supervisor–Specialist* còn phù hợp với LangGraph, nơi toàn bộ workflow được mô hình hóa thành đồ thị có trạng thái, hỗ trợ *interrupt–resume* cho *human-in-the-loop* và hỗ trợ vòng phản tư (*reflection loop*) khi hậu kiểm chất lượng yêu cầu sửa đổi. Do đó, đây là lựa chọn phù hợp hơn so với *single-agent* nếu mục tiêu không chỉ là “sinh được nội dung”, mà còn là quản lý được quá trình sinh nội dung một cách có thể giải thích và có thể kiểm chứng.

3.5.2 Giao thức giao tiếp và cơ chế điều phối giữa các tác tử

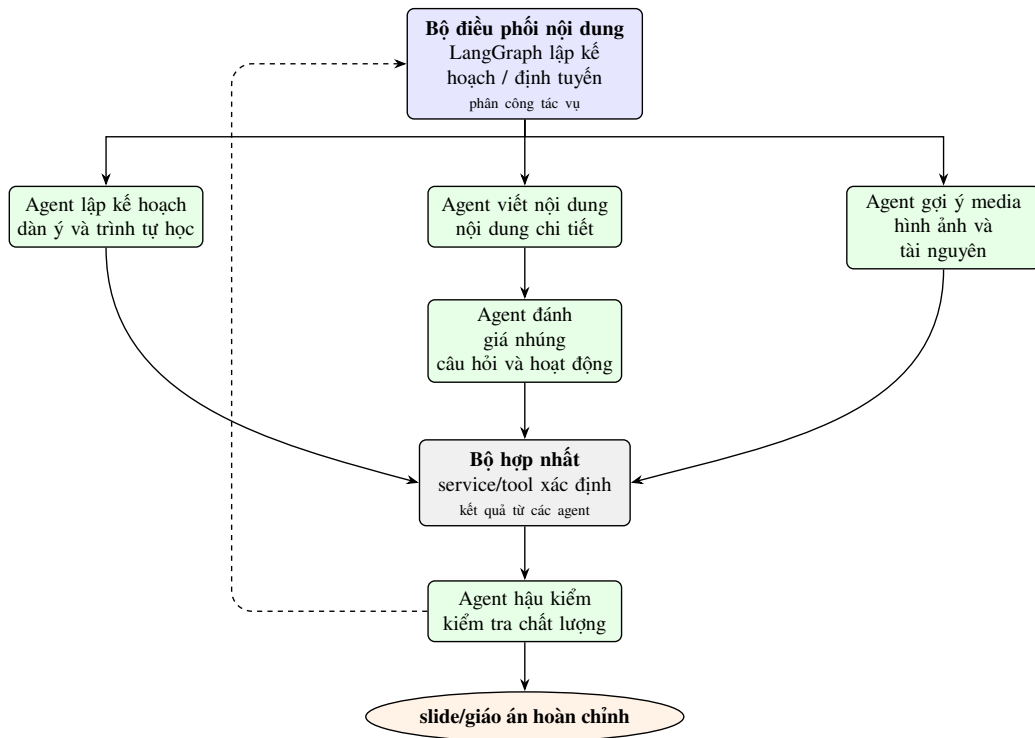
Để các tác tử chuyên biệt có thể phối hợp mà không phụ thuộc chặt vào cài đặt nội bộ của nhau, hệ thống sử dụng một giao thức trao đổi thống nhất dựa trên

hai thực thể *AgentTask* và *AgentTaskResult*. *AgentTask* là gói lệnh mà supervisor gửi cho một specialist agent, bao gồm định danh tác vụ, loại tác vụ, mục tiêu, đầu vào, tập ràng buộc, tham chiếu ngữ cảnh và loại artifact kỳ vọng. Chiều ngược lại, *AgentTaskResult* là gói phản hồi chuẩn hóa, chứa trạng thái thực thi, artifact đầu ra, điểm tin cậy (*confidence*), danh sách cảnh báo, thời gian xử lý và thông tin lỗi nếu có. Việc chuẩn hóa hai thực thể này giúp supervisor có thể điều phối nhiều agent bằng cùng một cơ chế, thay vì phải xây dựng một giao diện riêng cho từng agent.

Toàn bộ workflow được duy trì trên một *workflow state* dùng chung. Trạng thái này chứa không chỉ các trường đầu vào như `task_type`, `query`, `topic`, `grade`, `book` và tập `rag_chunks`, mà còn lưu các artifact trung gian như `outline_payload`, `content_payload`, `media_payload`, `quiz_payload`, `quality_review` và một *artifact registry* để supervisor có thể tra cứu kết quả theo `artifact_type`. Cấu trúc trạng thái này đóng vai trò là lớp bộ nhớ thao tác của riêng pipeline sinh nội dung, tách biệt với *session state* ở tầng điều phối đã trình bày trước đó.

Hình 3.7 mô tả quan hệ giữa supervisor, các specialist agent và bộ hợp nhất kết quả. Ở đỉnh sơ đồ là bộ điều phối nội dung, chịu trách nhiệm quyết định bước nào sẽ được gọi kế tiếp. Ở tầng dưới là các specialist agent tương ứng với các nhiệm vụ con: lập dàn ý, viết nội dung, gợi ý media và sinh đánh giá nhúng. Điểm cần nhấn mạnh là các agent này không giao tiếp trực tiếp với nhau; mọi yêu cầu đều đi qua supervisor. Cách tổ chức này giữ cho luồng điều khiển tập trung tại một điểm, nhờ đó dễ áp dụng các ràng buộc thứ tự như “phải có outline trước khi viết nội dung” hoặc “chỉ được hậu kiểm sau khi đã hợp nhất kết quả”.

Trong triển khai hiện tại, cơ chế điều phối được hiện thực bằng các *tool adapter* gắn với supervisor. Chúng vừa đóng vai trò cầu nối để chuyển từ quyết định của supervisor sang một *AgentTask* cụ thể, vừa đảm bảo rằng dữ liệu đầu vào cho từng specialist luôn tuân theo hợp đồng tương ứng. Chẳng hạn, bước `generate_content` sẽ bị chặn nếu chưa có `outline_payload`; bước `merge_results` chỉ được phép chạy sau khi ít nhất outline và content đã sẵn sàng; còn bước `check_quality` chỉ được kích hoạt sau khi artifact cuối đã được hợp nhất. Nhờ vậy, phần suy luận bằng mô hình ngôn ngữ được giữ trong phạm vi “quyết định dùng tác tử nào”, còn điều kiện hợp lệ của workflow vẫn được bảo vệ bằng logic xác định.



Hình 3.7: Kiến trúc đa tác tử cho bài toán sinh slide và giáo án

3.5.3 Các tác tử chuyên biệt trong pipeline sinh nội dung

Trong pipeline hiện tại, *PedagogyPlannerAgent* là tác tử được kích hoạt đầu tiên. Đầu vào của tác tử này là `context_map` đã được tổng hợp từ các chunk truy hồi, cùng với metadata về chủ đề, lớp, bộ sách và `task_type`. Nhiệm vụ của nó là xây dựng *outline payload* gồm tiêu đề bài học và danh sách các slide hoặc section dự kiến, mỗi phần tử có `slide_type`, tiêu đề, các ý chính và tham chiếu đến các chunk nguồn liên quan. Về bản chất, đây là bước chuyển từ ngữ cảnh truy hồi sang cấu trúc sơ phạm ở mức khung, tạo tiền đề cho các bước sinh nội dung sau đó.

ContentDraftingAgent là tác tử cốt lõi của pipeline vì nó trực tiếp biến outline thành nội dung giảng dạy. Tác tử này nhận `outline_payload` đã được duyệt cùng với `chunk_map`, sau đó sinh `content_payload` ở mức từng slide hoặc từng mục giáo án. Trong ràng buộc gửi kèm *AgentTask*, hệ thống đặt `must_use_source_chunk` do đó, nội dung sinh ra phải bám vào tập chunk đã truy hồi thay vì mở rộng tùy ý theo tri thức tham số của mô hình. Đây là cơ chế quan trọng để duy trì *grounding* cho các artifact có độ dài lớn hơn hỏi đáp ngắn.

MediaResearchAgent giữ vai trò hỗ trợ. Tác tử này không làm thay đổi luồng kiến thức chính của bài giảng mà chỉ sinh các gợi ý media như ảnh minh họa hoặc tài nguyên trực quan phù hợp với loại slide. Kết quả của bước này được lưu dưới dạng `media_payload`, gồm media cho trang mở đầu và media gắn với các slide nội dung. Trong trường hợp bước này thất bại, workflow vẫn có thể tiếp tục vì

media được xem là thành phần tăng cường chứ không phải điều kiện bắt buộc để hoàn thành artifact.

ContentAssessmentAgent chịu trách nhiệm sinh phần đánh giá nhúng (*embedded assessment*) thay vì tạo một bài kiểm tra độc lập. Đầu vào của tác tử này là chủ đề và ngữ cảnh bài học; đầu ra là `quiz_payload` chứa tập câu hỏi có thể gắn vào slide luyện tập hoặc vào phần cuối giáo án. Cách tổ chức này phù hợp với mục tiêu sử dụng thực tế của slide bài giảng, nơi thành phần đánh giá thường chỉ là một phần nhỏ trong artifact tổng thể. Tương tự media, nếu bước này không thành công thì supervisor vẫn có thể hoàn tất workflow bằng cách bỏ qua hoặc giảm nhẹ phần đánh giá.

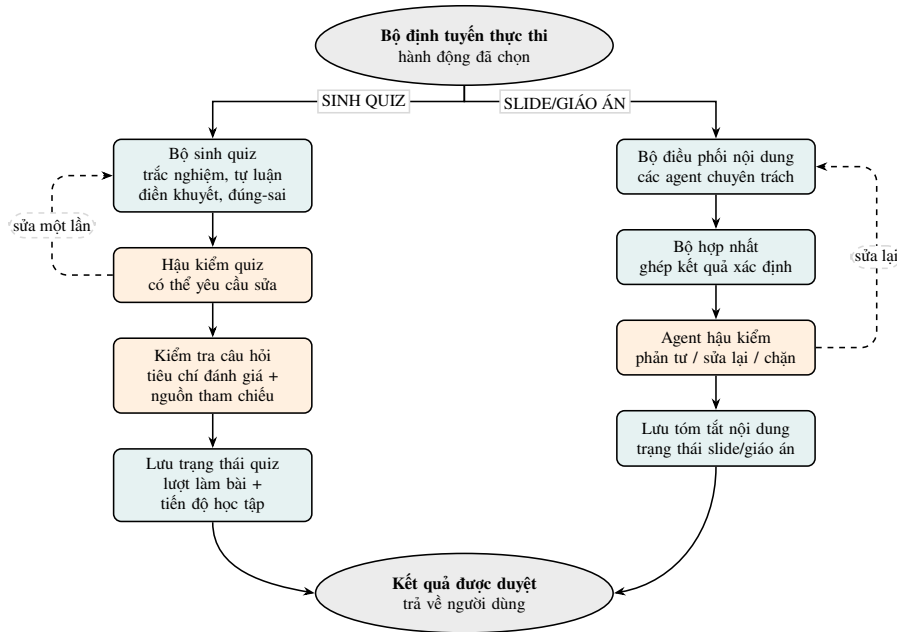
3.5.4 Tổng hợp kết quả, kiểm soát chất lượng và con người trong vòng lặp

Sau khi các specialist agent hoàn thành phần việc của mình, hệ thống không giao cho một mô hình ngôn ngữ khác nhiệm vụ “viết lại toàn bộ” artifact cuối. Thay vào đó, bước hợp nhất được thực hiện bởi một dịch vụ xác định (*deterministic service*) là *SlideMerger*. Dịch vụ này lấy `outline_payload` làm khung chuẩn, sau đó lần lượt ghép nội dung chi tiết, media và câu hỏi tương ứng vào từng slide theo `slide_id`. Cách hợp nhất này giúp bảo toàn cấu trúc đã được lập kế hoạch từ trước, đồng thời hạn chế nguy cơ một lượt gọi mô hình cuối cùng làm thay đổi ngoài ý muốn thứ tự hoặc nội dung của các phần đã được specialist tạo ra.

Về mặt chất lượng, hệ thống áp dụng hai lớp kiểm soát bổ sung nhau. Lớp thứ nhất là các kiểm tra cấu trúc mang tính xác định, chẳng hạn kiểm tra sự hiện diện của các loại slide tối thiểu, kiểm tra việc thiếu tiêu đề hoặc thiếu `source_chunk_ids` ở các slide cần grounding. Lớp thứ hai là *QualityReviewerAgent*, một tác tử hậu kiểm dùng mô hình ngôn ngữ để đánh giá artifact đầu ra theo rubric tổng quát và trả về `quality_review`. Kết quả hậu kiểm không chỉ gồm điểm số và các vấn đề phát hiện được, mà còn chỉ ra *reflection action* tương ứng, ví dụ chấp nhận, yêu cầu sửa outline, yêu cầu sửa content, yêu cầu sửa phần đánh giá hoặc chặn kết quả để cần người duyệt.

Hình 3.8 mô tả mối quan hệ giữa nhánh sinh nội dung và cơ chế hậu kiểm. Trong nhánh slide/giáo án, bộ điều phối nội dung gọi lần lượt các specialist, chuyển kết quả sang khối hợp nhất, rồi mới đưa artifact cuối qua lớp hậu kiểm chất lượng. Mũi tên nét đứt quay ngược từ khối hậu kiểm về bộ điều phối biểu diễn *reflection loop*: nếu reviewer xác định đầu ra cần chỉnh sửa, supervisor sẽ phát sinh một `revision_instruction` và kích hoạt lại đúng bước liên quan thay vì chạy lại toàn bộ pipeline từ đầu. Giới hạn số vòng phản tư được đặt nhỏ để kiểm soát chi phí và độ trễ.

Ngoài hậu kiểm tự động, pipeline còn tích hợp *human-in-the-loop* tại bước outline. Sau khi *PedagogyPlannerAgent* sinh dàn ý ban đầu, workflow có thể tạm dừng ở trạng thái *interrupt*. Người dùng được phép duyệt nguyên trạng hoặc chỉnh sửa outline trước khi supervisor tiếp tục gọi bước viết nội dung. Điểm chèn con người tại đây là hợp lý vì outline là nơi quyết định cấu trúc sơ phạm tổng thể; một chỉnh sửa sớm ở giai đoạn này thường hiệu quả hơn so với can thiệp sau khi toàn bộ slide hoặc giáo án đã được sinh xong.



Hình 3.8: Hai nhánh sinh nội dung và cơ chế hậu kiểm chất lượng của hệ thống

3.5.5 Tái sử dụng kiến trúc cho các tác vụ sinh nội dung

Một mục tiêu thiết kế quan trọng của tầng sinh nội dung là không xây dựng hai pipeline tách rời cho slide và giáo án. Thay vào đó, hệ thống dùng chung một kiến trúc đa tác tử, trong đó khác biệt giữa hai tác vụ được điều khiển chủ yếu bởi biến `task_type`. Giá trị này được truyền xuyên suốt từ đầu vào của *ContentPipelineInput*, vào *workflow state*, rồi đến từng *AgentTask*. Nhờ đó, cùng một supervisor và cùng một tập *specialist* có thể thích nghi đầu ra cho hai loại artifact mà không cần nhân đôi logic orchestration.

Việc tái sử dụng này thể hiện ở nhiều mức. Thứ nhất, cả hai tác vụ đều dùng chung tầng truy hồi, cùng cơ chế tổng hợp ngữ cảnh và cùng giao thức *AgentTask/AgentTaskResult*. Thứ hai, các *specialist* dùng chung hầu hết prompt và contract dữ liệu, chỉ thay đổi một số ràng buộc theo `task_type`, chẳng hạn slide thường yêu cầu ngắn gọn hơn và ưu tiên phần luyện tập nhúng, trong khi giáo án có thể cần cấu trúc section dài hơn. Thứ ba, bước hậu kiểm và phản tư vẫn giữ nguyên logic, chỉ thay reviewer hoặc rubric tương ứng với loại artifact cần đánh giá.

Khác biệt chủ yếu nằm ở giai đoạn đóng gói đầu ra. Với `task_type=slide`, artifact sau khi hợp nhất có thể được chuyển tiếp sang *SlideExportService* để xuất tệp PPTX. Với `task_type=lesson_plan`, pipeline vẫn dùng chung các bước sinh nội dung và hậu kiểm, nhưng không cần qua lớp xuất bản slide. Cách tổ chức này cho phép tái sử dụng phần lớn hạ tầng sinh nội dung, đồng thời vẫn giữ đủ linh hoạt để mở rộng thêm các loại artifact mới trong tương lai mà không phải thiết kế lại toàn bộ workflow.

3.6 Phương pháp đánh giá

3.6.1 Đánh giá truy hồi

3.6.2 Đánh giá chất lượng sinh

3.7 Kết chương

CHƯƠNG 4. CÀI ĐẶT, KIỂM THỬ VÀ KẾT QUẢ THỰC NGHIỆM

Chương 4 trình bày cách hiện thực hóa giải pháp đã đề xuất ở Chương 3, đồng thời báo cáo các kết quả kiểm thử và thực nghiệm đã thu được. Khác với Chương 3 vốn tập trung vào phương pháp, chương này chỉ giữ lại mức chi tiết cài đặt vừa đủ để chứng minh hệ thống đã được triển khai, benchmark đã được xây dựng và quy trình đánh giá đã có thể vận hành trên dữ liệu thực. Trọng tâm của chương là pipeline xử lý tài liệu, pipeline hỏi đáp, quy trình tạo benchmark, kiểm thử chức năng và các kết quả thực nghiệm hiện có đối với retrieval và generation.

4.1 Môi trường và dữ liệu thực nghiệm

Hệ thống được cài đặt chủ yếu bằng Python. Tầng điều phối nội dung cho tác vụ sinh slide và giáo án được xây dựng trên LangGraph, trong khi các thành phần còn lại như xử lý ngữ cảnh, định tuyến ý định, lập kế hoạch hành động, truy xuất tri thức và ghi vết thực thi được hiện thực bằng logic xác định trong mã nguồn. Cách tổ chức này giúp giữ chi phí suy luận ở mức hợp lý và tạo điều kiện thuận lợi cho việc phân tích lỗi theo từng bước của pipeline.

Về mô hình và hạ tầng tri thức, hệ thống sử dụng Google Gemini làm mô hình sinh ngôn ngữ, mô hình embedding tiếng Việt `dangvantuan/vietnamese-document-embedding` để biểu diễn chunk, mô hình reranker `AITeamVN/Vietnamese_Reranker` cho bước xếp hạng lại và Qdrant làm vector database. Bên cạnh *vector search*, hệ thống còn duy trì một kênh *keyword search* dựa trên BM25 tự cài đặt, sau đó hợp nhất hai danh sách kết quả bằng Reciprocal Rank Fusion. Các thành phần cài đặt chính của môi trường thực nghiệm được tóm tắt trong Bảng 4.1.

Thành phần	Cấu hình sử dụng
Ngôn ngữ lập trình	Python
Mô hình sinh ngôn ngữ	Google Gemini <code>gemini-2.5-flash-lite</code>
Mô hình embedding	<code>dangvantuan/vietnamese-document-embedding</code>
Mô hình reranker	<code>AITeamVN/Vietnamese_Reranker</code>
Vector database	Qdrant
Pipeline đa tác tử	LangGraph
Retriever lexical	BM25 tự cài đặt
Retriever ngữ nghĩa	Dense retrieval theo cosine similarity

Bảng 4.1: Môi trường phần mềm và các thành phần cốt lõi của hệ thống thực nghiệm

Nguồn dữ liệu của đồ án gồm SGK Tin học lớp 10, 11 và 12 thuộc hai bộ sách Cánh Diều và Kết Nối Tri Thức. Sau bước tiền xử lý và chunking, kho tri thức thu được 2348 chunk, mỗi chunk mang metadata về bộ sách, lớp, chủ đề, bài học, loại nội dung và cấp phân cấp. Chính cấu trúc này cho phép hệ thống vừa truy xuất ở mức cục bộ cho câu hỏi ngắn, vừa hỗ trợ các tác vụ cần bao phủ một bài học hoàn chỉnh như sinh slide hoặc soạn giáo án.

4.2 Cấu trúc cài đặt hệ thống

Tại mức cài đặt, hệ thống được chia thành bốn nhóm module chính. Nhóm thứ nhất là lớp tiếp nhận và điều phối yêu cầu, bao gồm các thành phần tạo *Request-Context*, phân tích ngữ cảnh hội thoại, định tuyến ý định, lập kế hoạch hành động và quản lý vòng đời session. Nhóm thứ hai là lớp dịch vụ nghiệp vụ, nơi triển khai các chức năng hỏi đáp, quiz, sinh slide và sinh giáo án. Nhóm thứ ba là lớp tri thức, phụ trách ingestion, chunking, embedding, indexing, retrieval, reranking và dựng ngữ cảnh đầu vào cho LLM. Nhóm cuối cùng là lớp thực nghiệm và đánh giá, gồm benchmark retrieval, kết quả generation theo RAGAS và các file raw output phục vụ phân tích lỗi.

Cách chia này nhằm giữ ranh giới rõ giữa phần điều phối hệ thống và phần nghiên cứu chính. Các module điều phối giúp hệ thống vận hành trên nhiều loại yêu cầu, nhưng phần đóng góp kỹ thuật được kiểm chứng trong chương này vẫn tập trung vào retrieval và generation. Do đó, Chương 4 không đi vào liệt kê toàn bộ package hay từng file cài đặt, mà chỉ trình bày đủ để người đọc hiểu dữ liệu đi qua những khối nào trước khi trở thành câu trả lời hoặc artifact cuối.

4.3 Cài đặt pipeline xử lý tài liệu

Pipeline xử lý tài liệu bắt đầu từ tập văn bản SGK đã được chuẩn hóa. Ở bước đầu, nội dung được làm sạch để loại bỏ nhiễu trình bày, sau đó được tách theo cấu trúc heading của từng bài học. Mỗi đoạn nội dung sau khi tách được gắn *breadcrumb*, *level* và metadata bài học tương ứng. Cách cài đặt này giúp chunk vẫn giữ được mối liên hệ với ngữ cảnh sư phạm của SGK, thay vì chỉ là các đoạn văn bản rời rạc.

Sau khi chunking, hệ thống sinh embedding trên trường `full_content` và nạp vector vào Qdrant. Song song với đó, các trường văn bản cần thiết cũng được lưu để phục vụ BM25 và để hiển thị nguồn khi phân tích kết quả retrieval. Việc lưu đồng thời cả metadata phân cấp lẫn vector embedding cho phép hệ thống lọc trước theo *scope* rồi mới thực hiện truy xuất, từ đó giảm rủi ro nhầm lẫn giữa các bài học có thuật ngữ gần nhau nhưng khác bộ sách hoặc khác khối lớp.

Dữ liệu tạo benchmark retrieval cũng được sinh trực tiếp từ kho chunk này. Mỗi

seed được gắn với một bài học đích, từ đó sinh các câu hỏi có *gold key* tương ứng để phục vụ đánh giá *single-gold*. Nhờ vậy, cùng một nền dữ liệu vừa đóng vai trò kho tri thức cho hệ thống, vừa là nguồn chuẩn để tạo bộ câu hỏi kiểm thử nhất quán với phạm vi dữ liệu của đồ án.

4.4 Cài đặt pipeline hỏi đáp

Khi người dùng gửi câu hỏi, hệ thống trước hết tạo một *RequestContext* để gom câu hỏi hiện tại, thông tin bộ sách hoặc lớp được chọn, lịch sử hội thoại gần và các trường debug cần thiết. Từ đây, *ContextAnalyzer* kiểm tra xem câu hỏi có phụ thuộc vào turn trước hay không; nếu có, hệ thống trích một phần ngữ cảnh hội thoại để làm giàu truy vấn. *IntentRouter* sau đó xác định loại yêu cầu, chẳng hạn hỏi đáp kiến thức, sinh quiz, sinh slide hoặc soạn giáo án.

Với các intent cần tri thức từ SGK, *ActionPlanner* tạo hành động tương ứng và chuyển cho *ExecutionDispatcher*. Ở nhánh hỏi đáp, dispatcher gọi *RAGService* để truy xuất ngữ cảnh, gửi ngữ cảnh này vào prompt sinh câu trả lời và ghi lại trace của toàn bộ phiên xử lý. Với các tác vụ dài như slide hoặc giáo án, dispatcher không sinh đầu ra trực tiếp mà chuyển tiếp cho pipeline đa tác tử đã mô tả ở Chương 3. Nhờ vậy, phần thực thi vẫn thống nhất ở tầng orchestration, trong khi logic chuyên biệt của từng tác vụ được cô lập ở các service tương ứng.

Điểm quan trọng trong cài đặt hiện tại là hệ thống không xem mọi yêu cầu như một lời gọi LLM đơn lẻ. Thay vào đó, truy vấn được biến đổi có kiểm soát qua các bước phân tích ngữ cảnh, định tuyến, retrieval, rồi mới tới generation. Cách tổ chức này làm rõ hơn quan hệ nhân quả giữa lỗi retrieval và lỗi answer, vốn là điều cần thiết cho các thí nghiệm báo cáo ở phần sau của chương.

4.5 Cài đặt retrieval, reranking và generation

Trong cấu hình retrieval dùng cho thực nghiệm hiện tại, hệ thống lấy *search pool* 50 candidate từ hai kênh lexical và dense, sau đó hợp nhất bằng RRF để thu được danh sách 25 candidate cuối cho bước đánh giá no-rerank. Ở cấu hình có rerank, cùng tập candidate này sẽ được chuyển qua cross-encoder để chấm lại mức độ liên quan trước khi chọn ra top-*N* cuối cùng. Cơ chế lọc theo *scope* được áp dụng trước truy xuất khi câu hỏi hoặc session đã xác định được *book*, *grade* hoặc *topic*.

Ở phía generation, prompt đầu vào luôn gồm hai thành phần: câu hỏi của người dùng và tập ngữ cảnh đã truy xuất. Hệ thống yêu cầu mô hình trả lời theo ngôn ngữ phù hợp với học sinh THPT, đồng thời ưu tiên bám sát thông tin xuất hiện trong context hơn là suy luận mở rộng từ tri thức tham số. Khi ngữ cảnh không đủ hoặc không truy xuất được tài liệu phù hợp, hệ thống được thiết kế để trả lời theo hướng nêu rõ giới hạn thay vì tiếp tục sinh nội dung không có căn cứ.

Việc tách riêng retrieval, reranking và generation trong cài đặt cho phép đánh giá từng bước một cách độc lập. Đây là tiền đề để phân kết quả retrieval ở Mục 4.8 và phần generation ở Mục 4.9 có thể được diễn giải như hai lớp bằng chứng khác nhau cho hiệu quả của hệ thống, thay vì trộn lẫn chúng trong một điểm số tổng hợp duy nhất.

4.6 Cài đặt benchmark và quy trình đánh giá

Bộ benchmark retrieval hiện tại được xây dựng theo hướng *single-gold*. Mỗi câu hỏi được gắn với một *lesson key* đích, gồm bộ sách, lớp, chủ đề, bài và tên bài học. Thay vì yêu cầu gán nhãn relevance chi tiết cho từng chunk, đồ án dùng lesson key làm chuẩn so khớp chính, nhờ đó việc tạo benchmark có thể mở rộng trên toàn bộ kho SGK mà vẫn giữ được tính nhất quán. Quá trình sinh benchmark bắt đầu từ 300 seed, cho ra 600 câu hỏi thô và bao phủ toàn bộ 185 bài học trong tập dữ liệu.

Sau bước sinh câu hỏi, benchmark được lọc bằng các tiêu chí xác định như khử trùng lặp, loại bỏ các câu mang tính template và kiểm tra rõ ràng tiêu đề ở mức n-gram. Kết quả lọc hiện tại không loại bỏ thêm mẫu nào, tức toàn bộ 600 câu đều đạt yêu cầu tối thiểu để đi tiếp vào pipeline đánh giá. Phân bố của bộ benchmark theo nhóm sách và khối lớp được trình bày trong Bảng 4.2; phân bố câu hỏi theo loại và độ khó cho thấy bộ câu hỏi thiên về các truy vấn định nghĩa, factual và application, phù hợp với hành vi hỏi đáp thực tế của học sinh.

Nhóm dữ liệu	Số lượng câu hỏi	Số bài được bao phủ
CD-10	100	185
CD-11	100	
CD-12	100	
KNTT-10	100	
KNTT-11	100	
KNTT-12	100	
Tổng cộng	600	185

Bảng 4.2: Phân bố bộ benchmark retrieval theo bộ sách và khối lớp

Đối với generation, đồ án tận dụng kết quả đánh giá RAGAS đã có sẵn trên 246 mẫu. Cách làm này phù hợp với điều kiện chi phí thực nghiệm: retrieval được đánh giá lại trực tiếp trên benchmark 600 câu hỏi mới, còn generation sử dụng lại tập đánh giá đã được chạy đầy đủ từ trước. Tại thời điểm viết báo cáo, luồng đánh giá with-rerank cho benchmark 600 câu hỏi vẫn chưa hoàn tất, vì vậy số liệu ở phần

retrieval mới chỉ phản ánh cấu hình no-rerank. Phần còn thiếu này được giữ mở để cập nhật sau mà không làm thay đổi cấu trúc chung của chương.

4.7 Kiểm thử chức năng

Ngoài đánh giá định lượng, hệ thống còn được kiểm thử ở mức chức năng để bảo đảm các bước cài đặt cơ sở vận hành đúng trước khi chạy benchmark. Các ca kiểm thử được chọn theo các điểm nút quan trọng của pipeline, từ ingestion dữ liệu, chunking, indexing đến scope filtering, retrieval, generation và trace logging. Bảng 4.3 tóm tắt các ca kiểm thử chính đã được sử dụng trong quá trình phát triển.

Mã	Chức năng	Kết quả mong đợi	Kết quả
TC01	Đọc dữ liệu SGK	Tài liệu được parse và làm sạch đúng định dạng đầu vào	Đạt
TC02	Chunking theo heading	Sinh chunk có metadata bài học và level nhất quán	Đạt
TC03	Embedding và indexing	Vector được tạo và nạp vào kho chỉ mục không lỗi	Đạt
TC04	Scope filtering	Truy xuất được giới hạn đúng theo sách hoặc lớp đã chọn	Đạt
TC05	Retrieval	Top- k chứa ngữ cảnh liên quan cho câu hỏi kiến thức	Đạt
TC06	Generation	Câu trả lời bám tài liệu và có thể ghi lại context debug	Đạt
TC07	Out-of-scope handling	Hệ thống ưu tiên báo thiếu thông tin thay vì bịa nội dung	Đạt
TC08	Multi-intent routing	Yêu cầu phức hợp được tách thành nhiều action hợp lệ	Đạt
TC09	Logging và trace	Mỗi request có debug step phục vụ phân tích lỗi	Đạt

Bảng 4.3: Các ca kiểm thử chức năng chính của hệ thống trước khi chạy thực nghiệm

Kết quả ở Bảng 4.3 cho thấy các khối nền tảng của hệ thống đã vận hành ổn định trước khi bước vào thực nghiệm. Đây là điều kiện cần để các số liệu retrieval và generation ở các mục tiếp theo có thể được diễn giải như kết quả của phương pháp, thay vì chỉ phản ánh lỗi cài đặt cơ bản của phần mềm.

4.8 Kết quả đánh giá retrieval

Thực nghiệm retrieval hiện đã được chạy đầy đủ trên 600 câu hỏi của benchmark *single-gold* trong cấu hình no-rerank. Mỗi câu hỏi được chấm tại các mức $k \in \{1, 3, 5, 10\}$ với các độ đo $\text{Hit}@k$, $\text{Recall}@k$, $\text{Precision}@k$, $\text{MRR}@k$ và $\text{nDCG}@k$. Vì benchmark sử dụng lesson key làm chuẩn đúng, các độ đo này phản ánh trực tiếp khả năng của retriever trong việc đưa đúng bài học mục tiêu vào top- k đầu tiên.

Bảng 4.4 cho thấy kết quả tổng thể của cấu hình no-rerank. Có thể thấy retriever đạt $\text{Hit}@1$ bằng 0,7617 và tăng lên 0,9817 ở $\text{Hit}@10$. $\text{MRR}@10$ đạt 0,8484 cho thấy ở phần lớn trường hợp, bài học đúng đã xuất hiện rất sớm trong danh sách kết quả. $\text{nDCG}@10$ đạt 0,4267 thấp hơn các độ đo Hit và MRR, điều này phù hợp với bản chất *single-gold*: mặc dù bài học đúng thường được truy xuất thành công, số lượng chunk liên quan trong cùng bài vẫn lớn nên việc phân bố thứ hạng bên trong lesson còn chưa tối ưu hoàn toàn.

Chỉ số	@1	@3	@5	@10
$\text{Hit}@k$	0,7617	0,9367	0,9683	0,9817
$\text{Recall}@k$	0,0684	0,1396	0,1879	0,2856
$\text{Precision}@k$	0,7617	0,5300	0,4350	0,3372
$\text{MRR}@k$	0,7617	0,8394	0,8467	0,8484
$\text{nDCG}@k$	0,7617	0,5790	0,5013	0,4267

Bảng 4.4: Kết quả retrieval tổng thể trên 600 câu hỏi benchmark ở cấu hình no-rerank

Để quan sát độ ổn định của retriever theo từng nhóm dữ liệu, Bảng 4.5 báo cáo ba độ đo đại diện là $\text{Hit}@1$, $\text{Hit}@10$ và $\text{MRR}@10$ theo từng cặp bộ sách – khối lớp. Kết quả cho thấy nhóm CD-11 đạt tốt nhất với $\text{Hit}@1$ bằng 0,90 và $\text{MRR}@10$ bằng 0,9367, trong khi CD-10 và KNTT-11 là hai nhóm khó hơn ở mức xếp hạng đầu. Dù vậy, tất cả các nhóm đều đạt $\text{Hit}@10$ từ 0,97 trở lên, cho thấy vấn đề chính không nằm ở việc bỏ sót hoàn toàn bài học đúng mà chủ yếu ở việc đẩy bài học đó lên đủ cao ở top đầu.

Nhóm dữ liệu	Hit@1	Hit@10	MRR@10
CD-10	0,7000	0,9700	0,8086
CD-11	0,9000	0,9900	0,9367
CD-12	0,7700	0,9800	0,8427
KNTT-10	0,7400	0,9800	0,8417
KNTT-11	0,7000	1,0000	0,8132
KNTT-12	0,7600	0,9700	0,8479

Bảng 4.5: Kết quả retrieval no-rerank theo từng nhóm bộ sách và khối lớp

Về thời gian xử lý, toàn bộ 600 câu hỏi được đánh giá trong khoảng 59,7 giây ở cấu hình no-rerank, tương đương trung bình xấp xỉ 0,10 giây cho mỗi truy vấn retrieval trong điều kiện chạy benchmark cục bộ. Con số này cho thấy cấu hình retriever hiện tại đủ nhẹ để mở rộng cho các thí nghiệm tiếp theo. Riêng cấu hình with-rerank vẫn chưa có số liệu hoàn chỉnh tại thời điểm viết báo cáo, do đó phần so sánh trước/sau rerank sẽ được cập nhật sau khi quá trình chạy đầy đủ trên toàn bộ benchmark hoàn tất.

4.9 Kết quả đánh giá generation và thảo luận

Đối với generation, đồ án sử dụng lại kết quả RAGAS đã được chạy trên 246 mẫu hỏi đáp. Các độ đo trung bình thu được gồm Faithfulness bằng 0,9757, Answer Relevancy bằng 0,8571, Context Precision bằng 0,8555 và Context Recall bằng 0,9593. Đồng thời, thời gian phản hồi trung bình của retriever là 4,579 giây, của bước sinh câu trả lời là 1,979 giây và của toàn bộ pipeline là 6,558 giây. Các số liệu này được tóm tắt trong Bảng 4.6.

Chỉ số đánh giá	Giá trị trung bình
Faithfulness	0,9757
Answer Relevancy	0,8571
Context Precision	0,8555
Context Recall	0,9593
Thời gian retriever (giây)	4,579
Thời gian generator (giây)	1,979
Tổng thời gian pipeline (giây)	6,558

Bảng 4.6: Kết quả đánh giá generation theo RAGAS trên 246 mẫu

Kết quả ở Bảng 4.6 cho thấy hai điểm đáng chú ý. Thứ nhất, *faithfulness* và

context recall đều ở mức cao, cho thấy khi retriever cung cấp được ngữ cảnh phù hợp, mô hình có xu hướng bám tốt vào nguồn và ít bịa thông tin ngoài tài liệu. Thứ hai, *answer relevancy* và *context precision* thấp hơn một bậc, phản ánh rằng chất lượng câu trả lời vẫn còn phụ thuộc mạnh vào việc sắp xếp đúng các chunk giá trị nhất ở đầu danh sách context. Nhận xét này cũng phù hợp với kết quả retrieval no-rerank ở Mục 4.8: bài học đúng thường đã được tìm thấy, nhưng thứ hạng giữa các chunk trong cùng lesson vẫn còn dư địa để cải thiện.

Từ hai nhóm kết quả trên, có thể rút ra rằng cấu hình hiện tại đã chứng minh được tính khả thi của hướng tiếp cận RAG trên dữ liệu SGK Tin học THPT. Retrieval ở mức lesson key hoạt động tốt ngay cả khi chưa áp dụng rerank đầy đủ trên benchmark 600 câu hỏi, trong khi generation đạt mức trung thực cao trên tập RAGAS đã có. Hạn chế lớn nhất tại thời điểm này là phần đối chứng with-rerank vẫn chưa hoàn tất, nên chưa thể kết luận định lượng mức cải thiện thực tế của cross-encoder trên benchmark mới. Đây cũng là hạng mục cần được bổ sung đầu tiên khi cập nhật lại chương thực nghiệm trong các bản sau.

Tóm lại, Chương 4 đã trình bày việc hiện thực hóa hệ thống từ dữ liệu SGK đến pipeline hỏi đáp hoàn chỉnh, đồng thời mô tả cách xây dựng benchmark và cách tổ chức đánh giá riêng cho retrieval và generation. Các kết quả hiện có cho thấy hệ thống đạt hiệu quả tốt ở cả khả năng truy xuất bài học đúng lẫn khả năng sinh câu trả lời bám nguồn. Phần kết quả với rerank chưa hoàn tất được giữ mở có chủ đích, nhằm bảo đảm rằng các kết luận trong chương chỉ dựa trên những số liệu đã thực sự được chạy và kiểm chứng.

CHƯƠNG 5. CÁC GIẢI PHÁP VÀ ĐÓNG GÓP NỔI BẬT

Các chương trước đã trình bày nền tảng lý thuyết và kết quả triển khai của hệ thống. Chương này tập trung phân tích các giải pháp và đóng góp nổi bật của đề án dưới góc nhìn thiết kế kỹ thuật. Trọng tâm không nằm ở việc liệt kê lại từng module, mà làm rõ các vấn đề khó đã gặp phải trong quá trình xây dựng trợ lý giáo dục dựa trên LLM và cách hệ thống giải quyết các vấn đề đó bằng kiến trúc có thể kiểm soát, dễ mở rộng và phù hợp với dữ liệu sách giáo khoa.

Các đóng góp chính gồm thiết kế điều phối request ở mức mã nguồn, cơ chế truy xuất thích nghi, nhận diện đa ý định, tách biệt pipeline quiz tương tác khỏi pipeline sinh nội dung dài, tổ chức content multi-agent cho slide và giáo án, chuẩn hóa giao tiếp nội bộ giữa các agent bằng A2A-lite, bổ sung HITL tại điểm quyết định quan trọng, và xây dựng các lớp kiểm tra chất lượng, session, trace phục vụ vận hành.

5.1 Điều phối request bằng thin orchestrator

5.1.1 Vấn đề cần giải quyết

Một hệ thống trợ lý giáo dục không chỉ nhận câu hỏi và gọi LLM để trả lời. Cùng một giao diện chat phải xử lý nhiều nhóm tác vụ khác nhau như hỏi đáp kiến thức, sinh câu hỏi luyện tập, chấm câu trả lời, ôn lại câu sai, tạo slide, tạo giáo án hoặc tiếp tục một pipeline đang chờ người dùng duyệt. Nếu đưa toàn bộ luồng xử lý vào một prompt lớn hoặc một agent tự quyết định mọi bước, hệ thống sẽ khó kiểm soát, khó gỡ lỗi và dễ gọi sai chức năng.

Vấn đề phức tạp hơn khi mỗi request cần mang theo nhiều loại trạng thái. Một câu hỏi có thể phụ thuộc lịch sử hội thoại, phạm vi sách/lớp người dùng chọn, chủ đề đã học trước đó, quiz state hoặc slide state đang chờ resume. Vì vậy, hệ thống cần một lớp điều phối rõ ràng để tách phần hiểu ngôn ngữ tự nhiên khỏi phần thực thi nghiệp vụ.

5.1.2 Giải pháp

Đề án thiết kế Orchestrator theo hướng thin controller. Orchestrator không chứa trực tiếp logic sinh câu hỏi, truy xuất tài liệu hoặc sinh slide, mà chủ yếu tạo `RequestContext`, phân tích ngữ cảnh, gọi `IntentRouter`, lập action plan và chuyển quyền thực thi cho `ExecutionDispatcher`. Cách làm này giữ Orchestrator ở vai trò điều phối, còn nghiệp vụ chi tiết được đóng gói trong các domain service hoặc handler.

`RequestContext` là state object đi xuyên qua toàn bộ vòng đời request. Nó lưu câu hỏi gốc, truy vấn đã làm giàu, danh sách query dùng cho RAG, intent,

action plan, phạm vi sách/lớp, session và debug steps. Nhờ đó, các module không cần truyền nhiều biến rời rạc hoặc phụ thuộc vào global state. Khi cần kiểm tra một lỗi, hệ thống có thể nhìn lại từng bước request đã đi qua.

Một điểm quan trọng là `ActionPlanner` hoạt động bằng rule-based logic thay vì gọi thêm LLM. LLM chỉ được dùng ở bước cần hiểu ngôn ngữ tự nhiên, còn ánh xạ từ intent sang action được giữ bằng mã nguồn tường minh. Điều này giảm độ trễ, tránh phát sinh nhánh xử lý ngoài kiểm soát và giúp việc thêm action mới có điểm mở rộng rõ ràng.

5.1.3 Đóng góp đạt được

Thiết kế này giải quyết được mâu thuẫn giữa tính linh hoạt của giao diện hội thoại và yêu cầu kiểm soát của hệ thống giáo dục. Người dùng vẫn có thể nhập yêu cầu tự nhiên, nhưng hệ thống không giao toàn bộ quyền điều phối cho LLM. Mỗi action được dispatch đến đúng service phụ trách, giúp kiến trúc dễ mở rộng hơn khi bổ sung chức năng mới.

Thin orchestrator cũng tạo nền tảng cho trace và debug. Mỗi request có request id, timestamp, debug steps và thông tin strategy RAG đã chọn. Đây là đóng góp thực tế quan trọng vì hệ thống LLM thường khó quan sát nếu chỉ nhìn đầu vào và đầu ra cuối cùng.

5.2 Truy xuất lại và Adaptive RAG

5.2.1 Vấn đề cần giải quyết

Dữ liệu sách giáo khoa có đặc điểm khác với dữ liệu web mở. Nhiều khái niệm quan trọng được trình bày ngắn gọn, có thuật ngữ chuyên ngành, tên lệnh, tên bài học hoặc cấu trúc chương mục rõ ràng. Nếu chỉ dùng tìm kiếm ngữ nghĩa, hệ thống có thể bỏ sót các đoạn có từ khóa chính xác. Nếu chỉ dùng tìm kiếm từ khóa, hệ thống khó xử lý câu hỏi diễn đạt tự nhiên khác với văn bản trong sách.

Ngoài ra, không phải câu hỏi nào cũng cần cùng một cách truy xuất. Câu hỏi “giải thích khái niệm cơ sở dữ liệu” cần đoạn nội dung chi tiết, trong khi câu hỏi “lớp 12 có những chủ đề nào” cần tổng hợp cấu trúc chương trình. Một pipeline RAG cố định dễ hoặc lấy thiếu ngữ cảnh, hoặc tốn chi phí không cần thiết.

5.2.2 Giải pháp

Hệ thống sử dụng truy xuất lai gồm tìm kiếm từ khóa BM25-like, tìm kiếm ngữ nghĩa bằng embedding tiếng Việt, hợp nhất thứ hạng bằng RRF và xếp hạng lại bằng reranker. BM25 giúp giữ độ nhạy với thuật ngữ chính xác, còn embedding giúp bắt được các câu hỏi diễn đạt theo cách tự nhiên. RRF được dùng để hợp nhất hai danh sách kết quả mà không yêu cầu điểm số của chúng nằm trên cùng một

thang đo.

Trên lớp truy xuất cơ sở, đồ án xây dựng AdaptiveRAGAgent. Thành phần này chọn strategy dựa trên QueryClassifier heuristic, không dùng LLM để ra quyết định. Bốn strategy chính gồm STANDARD, BROAD, CURRICULUM và HIERARCHICAL. STANDARD dùng cho truy vấn cụ thể nhưng thiếu phạm vi rõ. BROAD dùng cho câu hỏi tổng quan. CURRICULUM ưu tiên metadata chương trình học. HIERARCHICAL dùng khi có đủ tín hiệu lớp/chủ đề để truy xuất theo cấp bài học rồi mới đi vào chunk chi tiết.

Hệ thống còn bổ sung scope fallback. Khi phạm vi sách/lớp chỉ đến từ UI và không đủ chunk phù hợp, RAGService có thể mở rộng phạm vi tìm kiếm, đồng thời ghi lại actual_scope, scope_fallback_notice và debug step tương ứng. Nhờ vậy, hệ thống không trả lời rỗng quá sớm nhưng vẫn minh bạch về phạm vi dữ liệu đã dùng.

5.2.3 Đóng góp đạt được

Đóng góp ở đây không chỉ là ghép nhiều kỹ thuật retrieval, mà là đặt chúng trong một cơ chế chọn strategy theo loại câu hỏi. Cách làm này phù hợp với bài toán giáo dục vì câu hỏi của học sinh và giáo viên có mức độ cụ thể rất khác nhau. Adaptive RAG giúp hệ thống cân bằng giữa độ chính xác, độ bao phủ và chi phí truy xuất.

Việc QueryClassifier là heuristic cũng là một quyết định có chủ đích. Thay vì gọi LLM thêm một lượt để chọn chiến lược truy xuất, hệ thống giữ bước này nhanh, có thể kiểm thử và có thể giải thích. Đây là điểm quan trọng trong một hệ thống cần ổn định hơn là thể hiện khả năng suy luận tự do.

5.3 Multi-intent routing và action planning

5.3.1 Vấn đề cần giải quyết

Trong giao diện hội thoại, người dùng không luôn đưa ra một yêu cầu đơn lẻ. Một câu có thể chứa nhiều ý định, ví dụ yêu cầu tạo slide rồi thêm câu hỏi luyện tập, hoặc vừa giải thích kiến thức vừa sinh bài tập. Nếu hệ thống chỉ chọn một intent duy nhất, một phần yêu cầu sẽ bị bỏ qua. Ngược lại, nếu để LLM tự thực thi toàn bộ chuỗi hành động, hệ thống dễ sinh ra luồng không nhất quán với các service đang có.

5.3.2 Giải pháp

IntentRouter.detect_multi() phân tích tối đa ba intent trong một câu hỏi. Mỗi intent gồm nhóm ý định chính, loại nhiệm vụ, chủ đề, lớp, bộ sách và một số metadata cần thiết. Sau đó, ActionPlanner.plan_all() chuyển danh

sách intent thành danh sách action cụ thể như sinh quiz, sinh slide, sinh giáo án, chấm câu trả lời, ôn câu sai hoặc giải thích khái niệm.

Việc giới hạn số intent là một quyết định thực tế. Nó đủ để xử lý các câu lệnh ghép thường gặp, nhưng tránh để một câu nhập mơ hồ sinh ra quá nhiều action và làm phình state. Sau bước lập kế hoạch, `ExecutionDispatcher` chịu trách nhiệm gọi đúng service hoặc handler theo action đã xác định.

5.3.3 Đóng góp đạt được

Cơ chế này giúp hệ thống xử lý yêu cầu tự nhiên tốt hơn mà vẫn giữ được tính quyết định của backend. LLM được dùng để hiểu câu nói của người dùng, nhưng kế hoạch thực thi cuối cùng được đưa về cấu trúc action rõ ràng. Điều này tạo ra một ranh giới cần thiết giữa năng lực ngôn ngữ của LLM và logic nghiệp vụ của phần mềm.

5.4 Tách quiz standalone khỏi content assessment

5.4.1 Vấn đề cần giải quyết

Sinh câu hỏi luyện tập có thể xuất hiện trong hai bối cảnh khác nhau. Bối cảnh thứ nhất là quiz standalone, nơi người dùng tạo bộ câu hỏi, trả lời, được chấm điểm, xem lại câu sai và theo dõi tiến độ. Bối cảnh thứ hai là assessment nhúng trong slide hoặc giáo án, nơi câu hỏi chỉ là một artifact phụ để củng cố bài giảng.

Nếu gộp hai bối cảnh này vào cùng một agent hoặc cùng một pipeline, hệ thống sẽ khó quản lý state. Quiz standalone cần lưu từng lượt làm bài, từng câu hỏi, câu trả lời của người dùng, số lần thử và kết quả chấm. Assessment trong slide lại chủ yếu cần nội dung phù hợp với bài học và format trình bày.

5.4.2 Giải pháp

Đề án giữ `QuizService` là một service độc lập cho quiz tương tác. Service này quản lý vòng đời sinh câu hỏi, kiểm tra chất lượng, lưu `QuizRound`, lưu `QuestionRecord`, chấm câu trả lời, review câu sai và thống kê tiến độ. Các handler câu hỏi như trắc nghiệm, tự luận, điền khuyết và đúng/sai được giữ trong pipeline quiz riêng.

Trong content multi-agent pipeline, `ContentAssessmentAgent` chỉ sinh assessment nhúng cho slide hoặc giáo án. Agent này không thay thế `QuizService` và không quản lý state làm bài của học sinh. Cách tách này giúp tránh nhầm lẫn giữa một bài kiểm tra tương tác và một phần hoạt động trong tài liệu giảng dạy.

5.4.3 Đóng góp đạt được

Đây là một đóng góp kiến trúc quan trọng vì nó giữ đúng ranh giới nghiệp vụ. Quiz standalone có thể phát triển thêm các chức năng học tập cá nhân hóa mà không ảnh hưởng đến pipeline sinh slide. Ngược lại, pipeline sinh slide/giáo án có thể tối ưu cho artifact dài mà không phải mang theo toàn bộ logic chấm điểm và theo dõi attempt.

5.5 Content multi-agent cho slide và giáo án

5.5.1 Vấn đề cần giải quyết

Tạo slide và giáo án là tác vụ dài hơn nhiều so với hỏi đáp hoặc sinh một bộ câu hỏi ngắn. Hệ thống phải xác định mục tiêu học tập, lập dàn ý, viết nội dung chi tiết, gợi ý media, thêm assessment, ghép kết quả và kiểm tra chất lượng. Nếu giao toàn bộ tác vụ này cho một prompt duy nhất, đầu ra dễ thiếu cấu trúc, bỏ sót phần quan trọng hoặc khó sửa khi người dùng muốn thay đổi dàn ý.

5.5.2 Giải pháp

Đồ án dùng LangGraph cho phạm vi content pipeline của slide và giáo án. Ở cấp hệ thống, Orchestrator vẫn là controller mã nguồn; LangGraph chỉ điều phối luồng sinh nội dung dài. ContentSupervisor gọi các tool adapter tương ứng với các specialist agent như PedagogyPlannerAgent, ContentDraftingAgent, MediaResearchAgent, ContentAssessmentAgent và QualityReviewerAgent.

Các agent không trao đổi tự do mà đi qua contract A2A-lite. Supervisor tạo AgentTask, specialist agent trả AgentTaskResult. Task chứa mục tiêu, đầu vào, ràng buộc, context refs và artifact mong đợi. Kết quả chứa trạng thái, artifact, confidence, warnings, used tools và latency. Contract này giúp mỗi kết quả trung gian có chủ sở hữu rõ ràng và dễ ghi debug.

Sau khi các artifact được tạo, merge_results ghép outline, content, media và assessment thành kết quả thống nhất. Thành phần merge là service/tool quyết định, không phải agent. Sau đó, QualityReviewerAgent đánh giá kết quả cuối và reflection_decision_node quyết định approve, revise hoặc block theo số vòng reflection cho phép.

5.5.3 Đóng góp đạt được

Đóng góp chính của phần này là tổ chức tác vụ sinh nội dung dài thành các bước có artifact rõ ràng. Hệ thống không chỉ sinh một đoạn văn lớn, mà có dàn ý, nội dung, media metadata, assessment và review. Điều này giúp slide và giáo án dễ kiểm soát hơn, đồng thời chuẩn bị nền tảng để sau này xuất sang PPTX, PDF hoặc

HTML.

A2A-lite cũng là một lựa chọn phù hợp với quy mô hiện tại. Hệ thống chưa cần runtime distributed agent đầy đủ, nhưng đã có message envelope đủ rõ để quản lý task, artifact và metadata thực thi. Cách làm này giảm độ phức tạp vận hành nhưng vẫn giữ đường nâng cấp trong tương lai.

5.6 Human-in-the-loop tại bước duyệt dàn ý

5.6.1 Vấn đề cần giải quyết

Trong sinh slide và giáo án, dàn ý là quyết định ảnh hưởng đến toàn bộ artifact phía sau. Nếu dàn ý sai trọng tâm, thiếu mục hoặc chia bài không hợp lý, các bước sinh nội dung, media và assessment sau đó đều bị ảnh hưởng. Sửa ở cuối pipeline thường tốn chi phí hơn nhiều so với sửa ngay sau khi có outline.

5.6.2 Giải pháp

Hệ thống đặt HITL tại bước outline review. Sau khi agent lập dàn ý, graph có thể interrupt để người dùng duyệt hoặc gửi feedback chỉnh sửa. SlideService lưu các thông tin resume như thread id, graph config, trạng thái interrupt và task type vào slide state. Khi người dùng xác nhận hoặc góp ý, pipeline resume từ điểm đã dừng thay vì chạy lại toàn bộ request từ đầu.

5.6.3 Đóng góp đạt được

HITL giúp đưa quyền kiểm soát vào đúng điểm có ảnh hưởng lớn nhất. Người dùng, đặc biệt là giáo viên, có thể điều chỉnh cấu trúc bài giảng trước khi hệ thống sinh nội dung chi tiết. Cách làm này phù hợp với đặc thù giáo dục, nơi người dùng thường muốn kiểm soát mục tiêu và mạch bài học thay vì chỉ nhận một kết quả hoàn chỉnh.

5.7 Quality path, session và trace

5.7.1 Vấn đề cần giải quyết

Một hệ thống LLM phục vụ giáo dục cần nhiều lớp kiểm soát hơn một chatbot thông thường. Câu hỏi sinh ra cần đúng format và bám ngữ cảnh. Câu trả lời cần lưu được theo phiên để người dùng hỏi tiếp. Các lỗi cần có trace để biết hệ thống đã chọn intent, action và strategy RAG nào.

5.7.2 Giải pháp

Đồ án xây dựng hai quality path riêng. Với quiz standalone, QualityReviewer kiểm tra chất lượng bộ câu hỏi và QuestionValidator kiểm tra từng câu hỏi trước khi lưu vào session. Nếu cần, handler có thể được gọi lại với revision instruction. Với slide và giáo án, kết quả sau merge được đưa qua QualityRe-

`viewerAgent`; graph có thể approve, revise một phần hoặc block nếu chất lượng chưa đạt.

Session được lưu bằng `SessionStore` dưới dạng JSON. Session không chỉ lưu lịch sử chat mà còn lưu quiz state và slide state, cho phép các yêu cầu nối tiếp như “chấm câu 2”, “ôn câu sai” hoặc “ok, duyệt dần ý”. Song song với session, `TraceService` và `RequestContext.add_debug_step()` ghi lại các bước xử lý quan trọng để hỗ trợ kiểm thử và gỡ lỗi.

5.7.3 Đóng góp đạt được

Các lớp quality, session và trace làm cho hệ thống có khả năng vận hành thực tế hơn. Hệ thống không chỉ tạo được câu trả lời, mà còn quản lý được vòng đời học tập của người dùng và cung cấp dữ liệu quan sát khi kết quả chưa đạt kỳ vọng. Đây là nền tảng cần thiết để tiếp tục cải thiện chất lượng truy xuất, đánh giá và cá nhân hóa trong các phiên bản sau.

5.8 Tool layer như điểm mở rộng có kiểm soát

5.8.1 Vấn đề cần giải quyết

Một số năng lực của hệ thống không nên được nhúng trực tiếp vào prompt hoặc agent, ví dụ truy xuất tri thức, định dạng context, tìm kiếm media, kiểm tra URL hoặc xuất artifact. Nếu mỗi nơi gọi tool theo một kiểu riêng, hệ thống sẽ khó mở rộng và khó thay thế implementation.

5.8.2 Giải pháp

Đồ án xây dựng một MCP-like tool layer in-process. Lớp này gồm protocol request/response, client, server và registry tool. Các tool như knowledge retrieval hoặc content formatting có thể được gọi qua interface thống nhất. Hiện tại, lớp này chủ yếu đóng vai trò chuẩn hóa tool call nội bộ và làm extension point; chưa được mô tả như một MCP phân tán hoàn chỉnh.

5.8.3 Đóng góp đạt được

Tool layer giúp tách phần năng lực mở rộng khỏi core orchestration. Khi cần bổ sung media search thật, URL validation, source attribution hoặc export artifact, hệ thống đã có vị trí phù hợp để tích hợp mà không làm thay đổi sâu các service hiện có. Đây là một đóng góp về khả năng mở rộng hơn là một tính năng hoàn chỉnh ở thời điểm hiện tại.

5.9 Kết chương

Chương này đã hệ thống hóa các giải pháp và đóng góp chính của đồ án. Các đóng góp không nằm ở một thuật toán riêng lẻ, mà ở cách kết hợp LLM với các lớp điều phối, truy xuất, kiểm tra chất lượng, quản lý trạng thái và quan sát hệ thống.

Thiết kế này giúp Educational Chatbot tránh phụ thuộc hoàn toàn vào khả năng sinh tự do của LLM, đồng thời giữ được tính mở rộng cho các chức năng giáo dục như quiz, slide và giáo án.

Những đóng góp trên cũng cho thấy các khó khăn chính của đề tài đã được xử lý theo hướng thực dụng: truy xuất phải thích nghi với loại câu hỏi, điều phối phải kiểm soát được nhiều intent, quiz tương tác phải tách khỏi artifact generation, multi-agent phải có contract rõ ràng, và người dùng cần được đưa vào vòng duyệt ở các điểm quyết định quan trọng.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1 Kết luận

Sinh viên so sánh kết quả nghiên cứu hoặc sản phẩm của mình với các nghiên cứu hoặc sản phẩm tương tự.

Sinh viên phân tích trong suốt quá trình thực hiện ĐATN, mình đã làm được gì, chưa làm được gì, các đóng góp nổi bật là gì, và tổng hợp những bài học kinh nghiệm rút ra nếu có.

6.2 Hướng phát triển

Trong phần này, sinh viên trình bày định hướng công việc trong tương lai để hoàn thiện sản phẩm hoặc nghiên cứu của mình.

Trước tiên, sinh viên trình bày các công việc cần thiết để hoàn thiện các chức năng/nhiệm vụ đã làm. Sau đó sinh viên phân tích các hướng đi mới cho phép cải thiện và nâng cấp các chức năng/nhiệm vụ đã làm.

MỘT SỐ LƯU Ý VỀ TÀI LIỆU THAM KHẢO

Lưu ý: Sinh viên không được đưa bài giảng/slide, các trang Wikipedia, hoặc các trang web thông thường làm tài liệu tham khảo.

Một trang web được phép dùng làm tài liệu tham khảo **chỉ khi** nó là công bố chính thức của cá nhân hoặc tổ chức nào đó. Ví dụ, trang web đặc tả ngôn ngữ XML của tổ chức W3C <https://www.w3.org/TR/2008/REC-xml-20081126/> là TLTK hợp lệ.

Có năm loại tài liệu tham khảo mà sinh viên phải tuân thủ đúng quy định về cách thức liệt kê thông tin như sau. Lưu ý: các phần văn bản trong cặp dấu < > dưới đây chỉ là hướng dẫn khai báo cho từng loại tài liệu tham khảo; sinh viên cần xóa các phần văn bản này trong ĐATN của mình.

<**Bài báo đăng trên tạp chí khoa học:** Tên tác giả, tên bài báo, tên tạp chí, volume, từ trang đến trang (nếu có), nhà xuất bản, năm xuất bản >

[1] E. H. Hovy, "Automated discourse generation using discourse structure relations," *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993

<**Sách:** Tên tác giả, tên sách, volume (nếu có), lần tái bản (nếu có), nhà xuất bản, năm xuất bản>

[2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.

[3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.

<**Tập san Báo cáo Hội nghị Khoa học:** Tên tác giả, tên báo cáo, tên hội nghị, ngày (nếu có), địa điểm hội nghị, năm xuất bản>

[4] M. Poesio and B. Di Eugenio, "Discourse structure and anaphoric accessibility," in *ESSLLI workshop on information structure, discourse structure and discourse semantics*, Copenhagen, Denmark, 2001, pp. 129–143.

<**Đồ án tốt nghiệp, Luận văn Thạc sĩ, Tiến sĩ:** Tên tác giả, tên đồ án/luận văn, loại đồ án/luận văn, tên trường, địa điểm, năm xuất bản>

[5] A. Knott, "A data-driven methodology for motivating a set of coherence relations," Ph.D. dissertation, The University of Edinburgh, UK, 1996.

<**Tài liệu tham khảo từ Internet:** Tên tác giả (nếu có), tựa đề, cơ quan (nếu có), địa chỉ trang web, thời gian lần cuối truy cập trang web>

[6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. [Online]. Available:

`ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z` (visited on 09/30/2010).

[7] Princeton University, *Wordnet*. [Online]. Available: `http://www.cogsci.princeton.edu/~wn/index.shtml` (visited on 09/30/2010).

TÀI LIỆU THAM KHẢO

- [1] E. H. Hovy, “Automated discourse generation using discourse structure relations,” *Artificial intelligence*, vol. 63, no. 1-2, pp. 341–385, 1993.
- [2] L. L. Peterson and B. S. Davie, *Computer networks: a systems approach*. Elsevier, 2007.
- [3] N. T. Hải, *Mạng máy tính và các hệ thống mở*. Nhà xuất bản giáo dục, 1999.
- [4] M. Poesio and B. Di Eugenio, “Discourse structure and anaphoric accessibility,” in *ESSLLI workshop on information structure, discourse structure and discourse semantics, Copenhagen, Denmark*, 2001, pp. 129–143.
- [5] A. Knott, “A data-driven methodology for motivating a set of coherence relations,” Ph.D. dissertation, The University of Edinburgh, UK, 1996.
- [6] T. Berners-Lee, *Hypertext transfer protocol (HTTP)*. Accessed: Sep. 30, 2010. [Online]. Available: <ftp://info.cern.ch/pub/www/doc/http-spec.txt.Z>.
- [7] Princeton University, *Wordnet*. Accessed: Sep. 30, 2010. [Online]. Available: <http://www.cogsci.princeton.edu/~wn/index.shtml>.

PHỤ LỤC

A. HƯỚNG DẪN VIẾT ĐỒ ÁN TỐT NGHIỆP

Quy định chung

Dưới đây là một số quy định và hướng dẫn viết đồ án tốt nghiệp mà bắt buộc sinh viên phải đọc kỹ và tuân thủ nghiêm ngặt.

Sinh viên cần đảm bảo tính thống nhất toàn báo cáo (font chữ, căn dòng hai bên, hình ảnh, bảng, margin trang, đánh số trang, v.v.). Để làm được như vậy, sinh viên chỉ cần sử dụng các định dạng theo đúng template DATN này. Khi paste nội dung văn bản từ tài liệu khác của mình, sinh viên cần chọn kiểu Copy là “Text Only” để định dạng văn bản của template không bị phá vỡ/vi phạm.

Tuyệt đối cấm sinh viên đạo văn. Sinh viên cần ghi rõ nguồn cho tất cả những gì không tự mình viết/vẽ lên, bao gồm các câu trích dẫn, các hình ảnh, bảng biểu, v.v. Khi bị phát hiện, sinh viên sẽ không được phép bảo vệ DATN.

Tất cả các hình vẽ, bảng biểu, công thức, và tài liệu tham khảo trong DATN nhất thiết phải được SV giải thích và tham chiếu tới ít nhất một lần. Không chấp nhận các trường hợp sinh viên đưa ra hình ảnh, bảng biểu tùy hứng và không có lời mô tả/giải thích nào.

Sinh viên tuyệt đối không trình bày DATN theo kiểu viết ý hoặc gạch đầu dòng. DATN không phải là một slide thuyết trình; khi người đọc không hiểu sẽ không có ai giải thích hộ. Sinh viên cần viết thành các đoạn văn và phân tích, diễn giải đầy đủ, rõ ràng. Câu văn cần đúng ngữ pháp, đầy đủ chủ ngữ, vị ngữ và các thành phần câu. Khi thực sự cần liệt kê, sinh viên nên liệt kê theo phong cách khoa học với các ký tự La Mã. Ví dụ, nhiều sinh viên luôn cảm thấy hối hận vì (i) chưa cố gắng hết mình, (ii) chưa sắp xếp thời gian học/chơi một cách hợp lý, (iii) chưa tìm được người yêu để chia sẻ quãng đời sinh viên vất vả, và (iv) viết DATN một cách cẩu thả.

Trong một số trường hợp nhất thiết phải dùng các bullet để liệt kê, sinh viên cần thống nhất Style cho toàn bộ các bullet các cấp mà mình sử dụng đến trong báo cáo. Nếu dùng bullet cấp 1 là hình tròn đen, toàn bộ báo cáo cần thống nhất cách dùng như vậy; ví dụ như sau:

- Đây là mục 1 – Thực sự không còn cách nào khác tôi mới dùng đến việc bullet trong báo cáo.
- Đây là mục 2 – Nghĩ lại thì tôi có thể không cần dùng bullet cũng được. Nên tôi sẽ xóa bullet và tổ chức lại hai mục này trong báo cáo của mình cho khoa

học hơn. Tôi muốn thầy cô và người đọc cảm nhận được tâm huyết của tôi trong từng trang báo cáo DATN.

A.1 Ngành học

Sinh viên lưu ý viết đúng ngành/chuyên ngành trên bìa và trên gáy theo đúng quy định của Trường. Ngành học hay chuyên ngành học phụ thuộc vào ngành học mà sinh viên đăng ký. Sinh viên có thể đăng nhập trên trang quản lý học tập của mình để xem lại chính xác ngành học của mình.

Một số ví dụ sinh viên có thể tham khảo dưới đây, trong trường hợp có chuyên ngành thì sinh viên không cần ghi chuyên ngành:

1. Đối với kỹ sư chính quy:
 - (a) Từ K61 trở về trước: Ngành Kỹ thuật phần mềm
 - (b) Từ K62 trở về sau: Ngành Khoa học máy tính
2. Đối với cử nhân:
 - (a) Ngành Công nghệ thông tin
3. Đối với chương trình EliteTech:
 - (a) Chương trình Việt Nhật/KSTN: Ngành Công nghệ thông tin
 - (b) Chương trình ICT Global: Ngành Information Technology
 - (c) Chương trình DS&AI: Ngành Khoa học dữ liệu

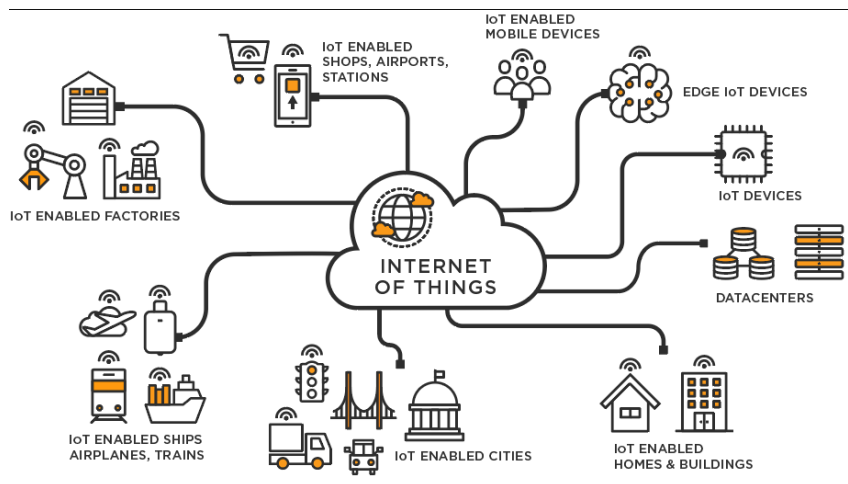
A.2 Đánh dấu (bullet) và đánh số (numering)

Việc sử dụng danh sách trong LaTeX khá đơn giản và không yêu cầu sinh viên phải thêm bất kỳ gói bổ sung nào. LaTeX cung cấp hai môi trường liệt kê đó là:

- Đánh dấu (bullet) là kiểu liệt kê không có thứ tự. Để sử dụng kiểu liệt kê đánh dấu, chúng ta khai báo như sau

```
\begin{itemize}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
\item ...
\end{itemize}
```
- Đánh số (numering) là kiểu liệt kê có thứ tự. Để sử dụng kiểu liệt kê đánh số, chúng ta khai báo như sau

```
\begin{enumerate}
\item Nội dung thứ nhất được viết ở đây.
\item Nội dung thứ hai được viết ở đây.
```



Hình A.1: Internet vạn vật

```
\item ...
\end{enumerate}
```

Chú ý các nội dung trình bày trong cả hai môi trường liệt kê theo sau lệnh `\item`. Ngoài ra LaTeX còn cung cấp một số kiểu liệt kê khác, sinh viên có thể tham khảo tại <https://www.overleaf.com/learn/latex/Lists>

A.3 Cách thêm bảng

Col1	Col2	Col2	Col3
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

Bảng A.1: Table to test captions and labels.

Bảng A.1 là ví dụ về cách tạo bảng. Tất cả các bảng biểu phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận. Chú ý: Tạo bảng trong LaTeX khá phức tạp và mất thời gian, vì vậy sinh viên có thể sử dụng các công cụ hỗ trợ tạo bảng (Ví dụ: <https://www.tablesgenerator.com/>). Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại link <https://www.overleaf.com/learn/latex/Tables>.

A.4 Chèn hình ảnh

Hình A.1 là ví dụ về cách chèn ảnh. Lưu ý chú thích của hình vẽ được đặt ngay dưới hình vẽ. Sinh viên có thể tìm hiểu sâu hơn về cách chèn ảnh trong LaTeX tại https://www.overleaf.com/learn/latex/Inserting_Images.

Chú ý, tất cả các hình vẽ phải được đề cập đến trong phần nội dung và phải được phân tích và bình luận.

A.5 Tài liệu tham khảo

Cách liệt kê

Áp dụng cách liệt kê theo quy định của IEEE. Ví dụ của việc trích dẫn như sau [2]. Cụ thể, sinh viên sử dụng lệnh `\cite{}` như sau [3]. Chỉ những tài liệu được trích dẫn thì mới xuất hiện trong phần Tài liệu tham khảo. Tài liệu tham khảo cần có nguồn gốc rõ ràng và phải từ nguồn đáng tin cậy. Hạn chế trích dẫn tài liệu tham khảo từ các website, từ wikipedia.

Các loại tài liệu tham khảo

Các nguồn tài liệu tham khảo chính là sách, bài báo trong các tạp chí, bài báo trong các hội nghị khoa học và các tài liệu tham khảo khác trên internet.

A.6 Cách viết phương trình và công thức toán học

Các gói `amsmath`, `amssymb`, `amsfonts` hỗ trợ viết phương trình/công thức toán học đã được bổ sung sẵn ở phần đầu của file `main.tex`. Một ví dụ về tạo phương trình (A.1) như sau

$$F(x) = \int_b^a \frac{1}{3}x^3 \quad (\text{A.1})$$

Phương trình A.1 là ví dụ về phương trình tích phân. Một phương trình khác không được đánh số thứ tự (gán nhãn)

$$x[t_n] = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} X[f_k] e^{j2\pi nk/N}$$

Phương trình này thể hiện phép biến đổi Fourier rời rạc ngược (IDFT).

A.7 Qui cách đóng quyển

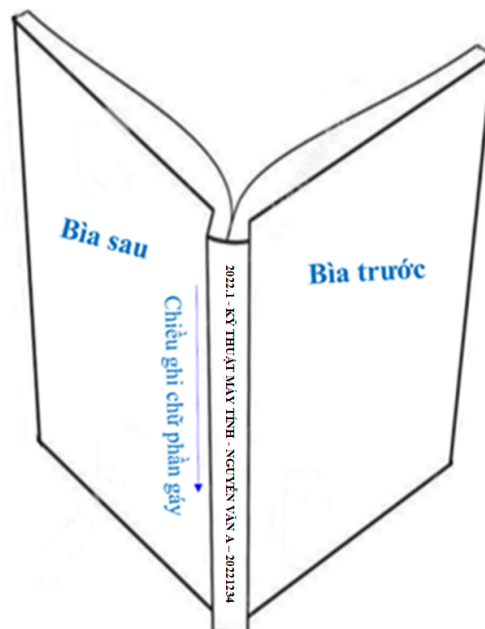
Phần bìa trước chế bản theo qui định; bìa trước và bìa sau là giấy liền khổ. Sử dụng keo nhiệt để dán gáy khi đóng quyển thay vì sử dụng băng dính và dập ghim như mô tả ở Hình A.2 Phần gáy ĐATN cần ghi các thông tin tóm tắt sau: Kỳ làm ĐATN - Ngành đào tạo - Họ và tên sinh viên - Mã số sinh viên. Ví dụ:

2022.1 - KỸ THUẬT MÁY TÍNH - NGUYỄN VĂN A - 20221234

Qui cách ghi chữ phần gáy như hình dưới đây:



Hình A.2: Qui cách đóng quyển đồ án



Hình A.3: Qui cách đóng quyển đồ án

B. ĐẶC TẢ USE CASE

Nếu trong nội dung chính không đủ không gian cho các use case khác (ngoài các use case nghiệp vụ chính) thì đặc tả thêm cho các use case đó ở đây.

B.1 Đặc tả use case “Tổng kê tình hình mượn sách”

...

B.2 Đặc tả use case “Đăng ký làm thẻ mượn”

...